

Barneshut

Generated by Doxygen 1.16.1

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 BoxBlurFilter Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 BoxBlurFilter()	8
4.1.3 Member Function Documentation	9
4.1.3.1 apply()	9
4.2 BrightnessFilter Class Reference	9
4.2.1 Detailed Description	10
4.2.2 Constructor & Destructor Documentation	10
4.2.2.1 BrightnessFilter() [1/2]	10
4.2.2.2 BrightnessFilter() [2/2]	11
4.2.3 Member Function Documentation	11
4.2.3.1 apply()	11
4.3 ConvolutionalFilter Class Reference	11
4.3.1 Detailed Description	13
4.3.2 Constructor & Destructor Documentation	13
4.3.2.1 ~ConvolutionalFilter()	13
4.3.3 Member Function Documentation	13
4.3.3.1 apply()	13
4.4 EmbossFilter Class Reference	13
4.4.1 Detailed Description	15
4.4.2 Constructor & Destructor Documentation	15
4.4.2.1 EmbossFilter()	15
4.4.3 Member Function Documentation	15
4.4.3.1 apply()	15
4.5 EqualizeHistogram Class Reference	16
4.5.1 Detailed Description	17
4.5.2 Constructor & Destructor Documentation	17
4.5.2.1 EqualizeHistogram()	17
4.5.3 Member Function Documentation	17
4.5.3.1 apply()	17
4.5.3.2 applyGrayscale()	18
4.5.3.3 applyRGB()	18

4.6 Filter3D Class Reference	18
4.6.1 Detailed Description	19
4.6.2 Constructor & Destructor Documentation	19
4.6.2.1 ~Filter3D()	19
4.6.3 Member Function Documentation	19
4.6.3.1 apply()	19
4.7 GaussianBlur3DFilter Class Reference	20
4.7.1 Detailed Description	21
4.7.2 Constructor & Destructor Documentation	21
4.7.2.1 GaussianBlur3DFilter()	21
4.7.3 Member Function Documentation	21
4.7.3.1 apply()	21
4.7.3.2 setIndex()	21
4.7.3.3 setPivot()	22
4.8 GaussianBlurFilter Class Reference	22
4.8.1 Detailed Description	23
4.8.2 Constructor & Destructor Documentation	23
4.8.2.1 GaussianBlurFilter()	23
4.8.3 Member Function Documentation	24
4.8.3.1 apply()	24
4.9 GreyscaleFilter Class Reference	24
4.9.1 Detailed Description	25
4.9.2 Member Function Documentation	25
4.9.2.1 apply()	25
4.10 HSLPixel Struct Reference	26
4.10.1 Detailed Description	26
4.10.2 Member Data Documentation	26
4.10.2.1 h	26
4.10.2.2 l	26
4.10.2.3 s	26
4.11 HSVPixel Struct Reference	27
4.11.1 Detailed Description	27
4.11.2 Member Data Documentation	27
4.11.2.1 h	27
4.11.2.2 s	27
4.11.2.3 v	27
4.12 Image Class Reference	28
4.12.1 Detailed Description	29
4.12.2 Constructor & Destructor Documentation	29
4.12.2.1 Image() [1/3]	29
4.12.2.2 Image() [2/3]	29
4.12.2.3 Image() [3/3]	30

4.12.3 Member Function Documentation	30
4.12.3.1 clone()	30
4.12.3.2 fromHSL()	30
4.12.3.3 fromHSV()	31
4.12.3.4 getChannels()	31
4.12.3.5 getData() [1/2]	31
4.12.3.6 getData() [2/2]	31
4.12.3.7 getHeight()	31
4.12.3.8 getPixel()	32
4.12.3.9 getPixelAsFloat()	32
4.12.3.10 getWidth()	32
4.12.3.11 HSLtoRGB()	33
4.12.3.12 HSVtoRGB()	33
4.12.3.13 load()	33
4.12.3.14 loadFromData()	34
4.12.3.15 removeAlpha()	34
4.12.3.16 RGBtoHSL()	34
4.12.3.17 RGBtoHSV()	35
4.12.3.18 save()	35
4.12.3.19 setChannels()	35
4.12.3.20 setPixel()	36
4.12.3.21 setPixelClamped()	36
4.12.3.22 stripAlphaChannel() [1/2]	36
4.12.3.23 stripAlphaChannel() [2/2]	36
4.12.3.24 toHSL()	37
4.12.3.25 toHSV()	37
4.13 MedianBlur3DFilter Class Reference	38
4.13.1 Detailed Description	39
4.13.2 Constructor & Destructor Documentation	39
4.13.2.1 MedianBlur3DFilter()	39
4.13.3 Member Function Documentation	39
4.13.3.1 apply()	39
4.14 MedianBlurFilter Class Reference	40
4.14.1 Detailed Description	41
4.14.2 Constructor & Destructor Documentation	41
4.14.2.1 MedianBlurFilter()	41
4.14.3 Member Function Documentation	41
4.14.3.1 apply()	41
4.15 PrewittFilter Class Reference	42
4.15.1 Detailed Description	43
4.15.2 Member Function Documentation	43
4.15.2.1 apply()	43

4.16 Projection Class Reference	43
4.16.1 Detailed Description	44
4.16.2 Member Function Documentation	44
4.16.2.1 averageIntensity()	44
4.16.2.2 maxIntensity()	44
4.16.2.3 medianIntensity()	45
4.16.2.4 minIntensity()	45
4.17 RGBPixel Struct Reference	46
4.17.1 Detailed Description	46
4.17.2 Member Data Documentation	46
4.17.2.1 b	46
4.17.2.2 g	46
4.17.2.3 r	46
4.18 RobertsCrossFilter Class Reference	47
4.18.1 Detailed Description	48
4.18.2 Member Function Documentation	48
4.18.2.1 apply()	48
4.19 SaltPepperFilter Class Reference	48
4.19.1 Detailed Description	49
4.19.2 Constructor & Destructor Documentation	49
4.19.2.1 SaltPepperFilter()	49
4.19.3 Member Function Documentation	50
4.19.3.1 apply()	50
4.20 ScharrFilter Class Reference	50
4.20.1 Detailed Description	51
4.20.2 Member Function Documentation	51
4.20.2.1 apply()	51
4.21 SharpenFilter Class Reference	52
4.21.1 Detailed Description	53
4.21.2 Member Function Documentation	53
4.21.2.1 apply()	53
4.22 SimpleFilter Class Reference	53
4.22.1 Detailed Description	54
4.22.2 Constructor & Destructor Documentation	54
4.22.2.1 ~SimpleFilter()	54
4.22.3 Member Function Documentation	54
4.22.3.1 apply()	54
4.23 Slice Class Reference	55
4.23.1 Detailed Description	55
4.23.2 Member Function Documentation	55
4.23.2.1 slice()	55
4.24 SliceVolume Class Reference	56

4.24.1 Detailed Description	57
4.24.2 Member Function Documentation	58
4.24.2.1 rotate()	58
4.24.2.2 slice()	58
4.25 SobelFilter Class Reference	58
4.25.1 Detailed Description	60
4.25.2 Member Function Documentation	60
4.25.2.1 apply()	60
4.26 ThresholdFilter Class Reference	60
4.26.1 Detailed Description	61
4.26.2 Constructor & Destructor Documentation	61
4.26.2.1 ThresholdFilter()	61
4.26.3 Member Function Documentation	62
4.26.3.1 apply()	62
4.27 Volume Class Reference	62
4.27.1 Detailed Description	64
4.27.2 Constructor & Destructor Documentation	64
4.27.2.1 Volume() [1/2]	64
4.27.2.2 Volume() [2/2]	64
4.27.2.3 ~Volume()	64
4.27.3 Member Function Documentation	64
4.27.3.1 clone()	64
4.27.3.2 getChannels()	65
4.27.3.3 getData() [1/2]	65
4.27.3.4 getData() [2/2]	65
4.27.3.5 getDepth()	65
4.27.3.6 getHeight()	65
4.27.3.7 getVoxel()	66
4.27.3.8 getVoxelAsFloat()	66
4.27.3.9 getWidth()	66
4.27.3.10 load()	67
4.27.3.11 setVoxel() [1/2]	67
4.27.3.12 setVoxel() [2/2]	67
4.27.3.13 setVoxelClamped() [1/2]	68
4.27.3.14 setVoxelClamped() [2/2]	68
4.27.4 Member Data Documentation	69
4.27.4.1 channels	69
4.27.4.2 data	69
4.27.4.3 depth	69
4.27.4.4 height	69
4.27.4.5 width	69

5 File Documentation	71
5.1 ConvolutionalFilter.cpp File Reference	71
5.1.1 Detailed Description	72
5.1.2 Function Documentation	72
5.1.2.1 calculateGaussianKernel()	72
5.1.2.2 EdgeDetection()	72
5.2 ConvolutionalFilter.h File Reference	72
5.2.1 Detailed Description	73
5.3 ConvolutionalFilter.h	74
5.4 Filters3D.cpp File Reference	75
5.4.1 Detailed Description	75
5.5 Filters3D.h File Reference	76
5.5.1 Detailed Description	77
5.6 Filters3D.h	77
5.7 Image.cpp File Reference	78
5.7.1 Detailed Description	78
5.8 Image.h File Reference	79
5.8.1 Detailed Description	80
5.9 Image.h	80
5.10 main.cpp File Reference	81
5.10.1 Detailed Description	82
5.10.2 Macro Definition Documentation	83
5.10.2.1 STB_IMAGE_IMPLEMENTATION	83
5.10.2.2 STB_IMAGE_WRITE_IMPLEMENTATION	83
5.10.3 Function Documentation	83
5.10.3.1 main()	83
5.11 Projection.cpp File Reference	83
5.11.1 Detailed Description	84
5.12 Projection.h File Reference	84
5.12.1 Detailed Description	85
5.13 Projection.h	85
5.14 SimpleFilter.cpp File Reference	86
5.14.1 Detailed Description	86
5.15 SimpleFilter.h File Reference	87
5.15.1 Detailed Description	88
5.16 SimpleFilter.h	88
5.17 Slice.cpp File Reference	89
5.17.1 Detailed Description	90
5.18 Slice.h File Reference	90
5.18.1 Detailed Description	91
5.19 Slice.h	92
5.20 SliceVolume.cpp File Reference	92

5.20.1 Detailed Description	93
5.21 SliceVolume.h File Reference	93
5.21.1 Detailed Description	94
5.22 SliceVolume.h	95
5.23 Volume.cpp File Reference	95
5.23.1 Detailed Description	96
5.24 Volume.h File Reference	96
5.24.1 Detailed Description	97
5.25 Volume.h	97
Index	99

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Filter	
ConvolutionalFilter	11
BoxBlurFilter	7
EmbossFilter	13
GaussianBlurFilter	22
MedianBlurFilter	40
PrewittFilter	42
RobertsCrossFilter	47
ScharFilter	50
SharpenFilter	52
SobelFilter	58
SimpleFilter	53
BrightnessFilter	9
EqualizeHistogram	16
GreyscaleFilter	24
SaltPepperFilter	48
ThresholdFilter	60
Filter3D	18
GaussianBlur3DFilter	20
MedianBlur3DFilter	38
HSLPixel	26
HSVPixel	27
Image	28
Projection	43
RGBPixel	46
Slice	55
Volume	62
SliceVolume	56

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BoxBlurFilter	Box blur (mean blur) filter. Smooths the image by replacing each pixel with the average value of its local neighborhood	7
BrightnessFilter	Adjusts the brightness of an image	9
ConvolutionalFilter	Base class for convolutional filters. Inherits from Filter and provides an abstract interface for all image processing operations based on convolution matrices (kernels)	11
EmbossFilter	Emboss effect filter. Creates a 3D embossed visual effect by replacing pixels with shadows cast from a specific direction	13
EqualizeHistogram	Performs histogram equalization to improve image contrast	16
Filter3D	Base class for separable 3D convolution filters	18
GaussianBlur3DFilter	3D Gaussian blur filter	20
GaussianBlurFilter	Gaussian blur filter. Uses a Gaussian function to calculate weights for smoothing, which preserves edge details better than a simple box blur	22
GreyscaleFilter	Converts a color image to grayscale	24
HSLPixel	Represents a single pixel in the HSL (Hue, Saturation, Lightness) colour space	26
HSVPixel	Represents a single pixel in the HSV (Hue, Saturation, Value) colour space	27
Image	A concrete class for storing and manipulating image data	28
MedianBlur3DFilter	3D Median blur filter	38
MedianBlurFilter	Median blur filter. Replaces each pixel with the median value of its neighboring pixels. Highly effective for removing salt-and-pepper noise	40
PrewittFilter	Prewitt edge detection filter. Uses the Prewitt operator for edge detection. Suitable for images with noticeable edges and some noise	42

Projection	A utility class for generating 2D image projections from a 3D volume	43
RGBPixel	Represents a single pixel in the RGB colour space	46
RobertsCrossFilter	Roberts Cross edge detection filter. Approximates gradient magnitude using diagonal directional differences. Computationally fast	47
SaltPepperFilter	Injects random salt-and-pepper noise into an image	48
ScharrFilter	Scharr edge detection filter. Provides a more accurate, rotation-invariant gradient calculation compared to the Sobel operator	50
SharpenFilter	Sharpen filter. Enhances the edges and detail contrast of the image	52
SimpleFilter	Abstract base class for simple, point-process image filters	53
Slice	A utility class for extracting 2D images from a 3D volume	55
SliceVolume	Represents a 3D volume that supports plane rotation and slice extraction	56
SobelFilter	Sobel edge detection filter. Highlights edges by calculating the spatial gradient of the image intensity using the Sobel operator	58
ThresholdFilter	Applies a threshold to an image, converting it to binary (black and white)	60
Volume	A class representing a 3D volume of pixel/voxel data	62

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

ConvolutionalFilter.cpp	Implementation of various convolutional image filters	71
ConvolutionalFilter.h	Defines a collection of convolutional image filters	72
Filters3D.cpp	Implementation of 3D volumetric image filters	75
Filters3D.h	Defines 3D volumetric image filters	76
Image.cpp	Implementation of the Image class for image manipulation and I/O	78
Image.h	Defines the Image class and colour-space pixel structures	79
main.cpp	Main entry point for the APLImageFilters application	81
Projection.cpp	Implementation of 2D image projections from 3D volumetric data	83
Projection.h	Defines operations for creating 2D projections from 3D volumetric data	84
SimpleFilter.cpp	Implementation of simple point-process image filters	86
SimpleFilter.h	Defines a collection of simple point-process image filters	87
Slice.cpp	Implementation of 2D slice extraction from 3D volumetric data	89
Slice.h	Defines operations for extracting 2D slices from 3D volumetric data	90
SliceVolume.cpp	Implementation of the SliceVolume class for optimized slice extraction	92
SliceVolume.h	Defines a specialized Volume class for slice manipulation	93
Volume.cpp	Implementation of the Volume class for 3D volumetric data management	95
Volume.h	Defines the base Volume class for 3D volumetric data	96

Chapter 4

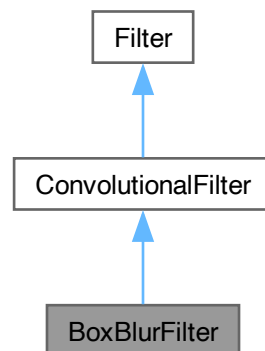
Class Documentation

4.1 BoxBlurFilter Class Reference

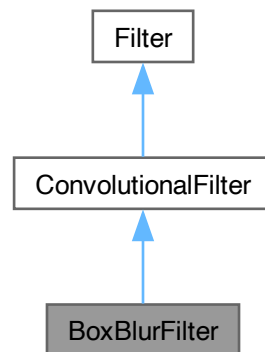
Box blur (mean blur) filter. Smooths the image by replacing each pixel with the average value of its local neighborhood.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for BoxBlurFilter:



Collaboration diagram for BoxBlurFilter:



Public Member Functions

- [BoxBlurFilter](#) (int size)
Constructs a [BoxBlurFilter](#) object.
- void [apply](#) ([Image](#) &image) override
Applies the box blur operation to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.1.1 Detailed Description

Box blur (mean blur) filter. Smooths the image by replacing each pixel with the average value of its local neighborhood.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 BoxBlurFilter()

```
BoxBlurFilter::BoxBlurFilter (  
    int size)
```

Constructs a [BoxBlurFilter](#) object.

Parameters

<i>size</i>	The size of the kernel (usually an odd number, e.g., 3 for 3x3).
-------------	--

4.1.3 Member Function Documentation

4.1.3.1 `apply()`

```
void BoxBlurFilter::apply (  
    Image & image) [override], [virtual]
```

Applies the box blur operation to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

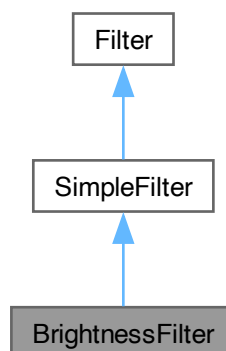
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.2 BrightnessFilter Class Reference

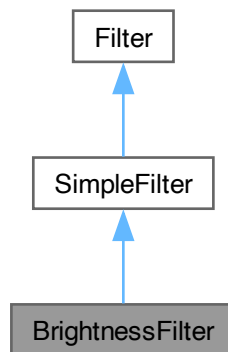
Adjusts the brightness of an image.

```
#include <SimpleFilter.h>
```

Inheritance diagram for BrightnessFilter:



Collaboration diagram for BrightnessFilter:



Public Member Functions

- [BrightnessFilter](#) (int value)
Constructs a [BrightnessFilter](#) with a specific manual adjustment value.
- [BrightnessFilter](#) ()
Constructs a [BrightnessFilter](#) in automatic mode.
- void [apply](#) ([Image](#) &image) override
Applies the brightness adjustment to the image.

Public Member Functions inherited from [SimpleFilter](#)

- virtual [~SimpleFilter](#) ()
Default virtual destructor.

4.2.1 Detailed Description

Adjusts the brightness of an image.

Supports both a manual adjustment mode (by a specific value) and an automatic adjustment mode.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 [BrightnessFilter](#)() [1/2]

```
BrightnessFilter::BrightnessFilter (  
    int value)
```

Constructs a [BrightnessFilter](#) with a specific manual adjustment value.

Parameters

<i>value</i>	The brightness offset (positive to brighten, negative to darken).
--------------	---

4.2.2.2 BrightnessFilter() [2/2]

```
BrightnessFilter::BrightnessFilter ()
```

Constructs a [BrightnessFilter](#) in automatic mode.

4.2.3 Member Function Documentation**4.2.3.1 apply()**

```
void BrightnessFilter::apply (  
    Image & image) [override], [virtual]
```

Applies the brightness adjustment to the image.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

Implements [SimpleFilter](#).

The documentation for this class was generated from the following files:

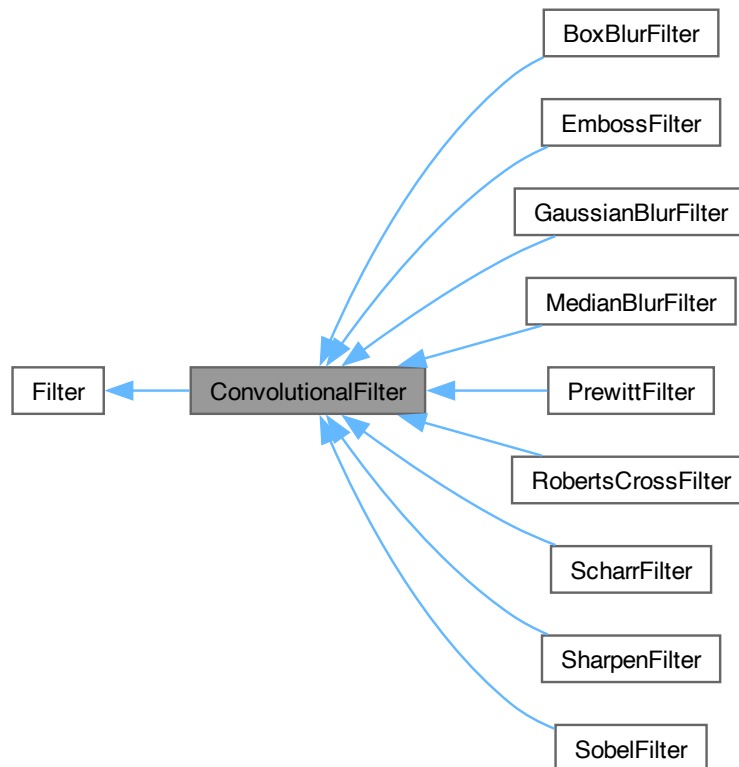
- [SimpleFilter.h](#)
- [SimpleFilter.cpp](#)

4.3 ConvolutionalFilter Class Reference

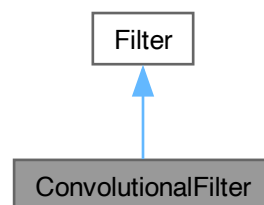
Base class for convolutional filters. Inherits from [Filter](#) and provides an abstract interface for all image processing operations based on convolution matrices (kernels).

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for ConvolutionalFilter:



Collaboration diagram for ConvolutionalFilter:



Public Member Functions

- virtual `~ConvolutionalFilter()`
Default virtual destructor.
- virtual void `apply(Image &image)` override=0
Applies the convolutional filter to the target image.

4.3.1 Detailed Description

Base class for convolutional filters. Inherits from `Filter` and provides an abstract interface for all image processing operations based on convolution matrices (kernels).

4.3.2 Constructor & Destructor Documentation

4.3.2.1 `~ConvolutionalFilter()`

```
virtual ConvolutionalFilter::~ConvolutionalFilter () [inline], [virtual]
```

Default virtual destructor.

4.3.3 Member Function Documentation

4.3.3.1 `apply()`

```
virtual void ConvolutionalFilter::apply (
    Image & image) [override], [pure virtual]
```

Applies the convolutional filter to the target image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implemented in [BoxBlurFilter](#), [EmbossFilter](#), [GaussianBlurFilter](#), [MedianBlurFilter](#), [PrewittFilter](#), [RobertsCrossFilter](#), [ScharrFilter](#), [SharpenFilter](#), and [SobelFilter](#).

The documentation for this class was generated from the following file:

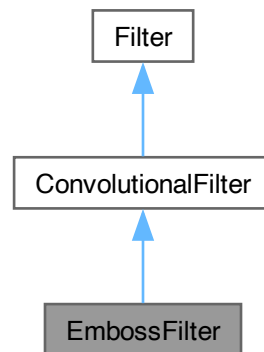
- [ConvolutionalFilter.h](#)

4.4 EmbossFilter Class Reference

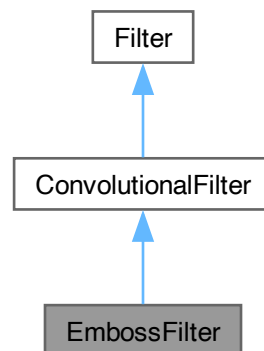
Emboss effect filter. Creates a 3D embossed visual effect by replacing pixels with shadows cast from a specific direction.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for EmbossFilter:



Collaboration diagram for EmbossFilter:



Public Member Functions

- [EmbossFilter](#) (float strength, const std::string &direction, const std::string &type="HSV")
Constructs an [EmbossFilter](#) object.
- void [apply](#) ([Image](#) &image) override
Applies the emboss effect to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.4.1 Detailed Description

Emboss effect filter. Creates a 3D embossed visual effect by replacing pixels with shadows cast from a specific direction.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 EmbossFilter()

```
EmbossFilter::EmbossFilter (  
    float strength,  
    const std::string & direction,  
    const std::string & type = "HSV")
```

Constructs an [EmbossFilter](#) object.

Parameters

<i>strength</i>	The strength of the effect.
<i>direction</i>	The light source direction (e.g., "NW" for North-West).
<i>type</i>	The color space to use. Defaults to "HSV".

4.4.3 Member Function Documentation

4.4.3.1 apply()

```
void EmbossFilter::apply (  
    Image & image) [override], [virtual]
```

Applies the emboss effect to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

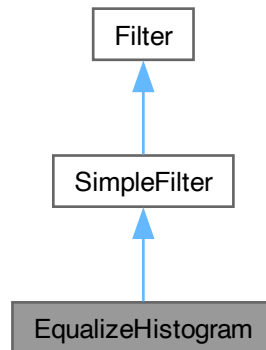
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.5 EqualizeHistogram Class Reference

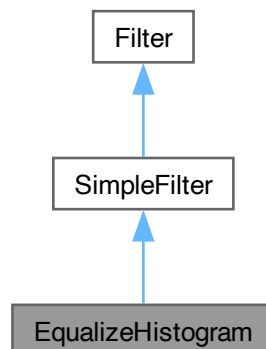
Performs histogram equalization to improve image contrast.

```
#include <SimpleFilter.h>
```

Inheritance diagram for EqualizeHistogram:



Collaboration diagram for EqualizeHistogram:



Public Member Functions

- [EqualizeHistogram](#) (const std::string &type="HSV")
Constructs an [EqualizeHistogram](#) filter.
- void [apply](#) ([Image](#) &image) override
Applies the histogram equalization to the image automatically determining type.

- void [applyRGB](#) ([Image](#) &image)
Explicitly applies histogram equalization assuming an RGB format.
- void [applyGrayscale](#) ([Image](#) &image)
Explicitly applies histogram equalization assuming a grayscale format.

Public Member Functions inherited from [SimpleFilter](#)

- virtual [~SimpleFilter](#) ()
Default virtual destructor.

4.5.1 Detailed Description

Performs histogram equalization to improve image contrast.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 EqualizeHistogram()

```
EqualizeHistogram::EqualizeHistogram (
    const std::string & type = "HSV")
```

Constructs an [EqualizeHistogram](#) filter.

Parameters

<i>type</i>	The color model to use for processing (defaults to "HSV").
-------------	--

4.5.3 Member Function Documentation

4.5.3.1 apply()

```
void EqualizeHistogram::apply (
    Image & image) [override], [virtual]
```

Applies the histogram equalization to the image automatically determining type.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

Implements [SimpleFilter](#).

4.5.3.2 applyGrayscale()

```
void EqualizeHistogram::applyGrayscale (  
    Image & image)
```

Explicitly applies histogram equalization assuming a grayscale format.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

4.5.3.3 applyRGB()

```
void EqualizeHistogram::applyRGB (  
    Image & image)
```

Explicitly applies histogram equalization assuming an RGB format.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

The documentation for this class was generated from the following files:

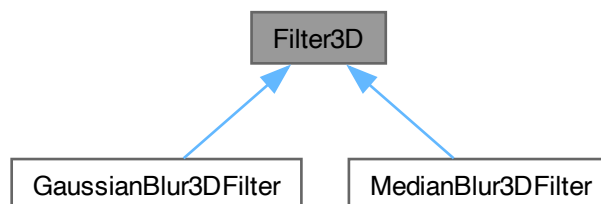
- [SimpleFilter.h](#)
- [SimpleFilter.cpp](#)

4.6 Filter3D Class Reference

Base class for separable 3D convolution filters.

```
#include <Filters3D.h>
```

Inheritance diagram for Filter3D:



Public Member Functions

- virtual [~Filter3D](#) ()
Default virtual destructor.
- virtual void [apply](#) ([Volume](#) &volume)=0
Applies the 3D filter to the given volume.

4.6.1 Detailed Description

Base class for separable 3D convolution filters.

Provides an abstract interface for applying 3D filters to volumetric data.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 ~Filter3D()

```
virtual Filter3D::~~Filter3D () [inline], [virtual]
```

Default virtual destructor.

4.6.3 Member Function Documentation

4.6.3.1 apply()

```
virtual void Filter3D::apply (  
    Volume & volume) [pure virtual]
```

Applies the 3D filter to the given volume.

Parameters

<i>volume</i>	Reference to the 3D volume object to be processed in-place.
---------------	---

Implemented in [GaussianBlur3DFilter](#), and [MedianBlur3DFilter](#).

The documentation for this class was generated from the following file:

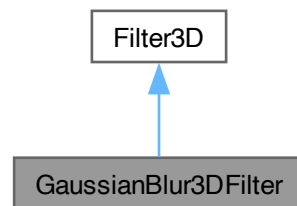
- [Filters3D.h](#)

4.7 GaussianBlur3DFilter Class Reference

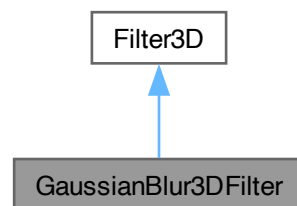
3D Gaussian blur filter.

```
#include <Filters3D.h>
```

Inheritance diagram for GaussianBlur3DFilter:



Collaboration diagram for GaussianBlur3DFilter:



Public Member Functions

- [GaussianBlur3DFilter](#) (int size, double stdev)
Constructs a [GaussianBlur3DFilter](#) object.
- void [apply](#) ([Volume](#) &volume) override
Applies the 3D Gaussian blur operation to the volume.
- void [setPivot](#) (char pivot)
Sets the pivot axis for the separable convolution.
- void [setIndex](#) (int axIndex)
Sets the axis index for the convolution pass.

Public Member Functions inherited from [Filter3D](#)

- virtual [~Filter3D](#) ()
Default virtual destructor.

4.7.1 Detailed Description

3D Gaussian blur filter.

Applies a separable 3D Gaussian blur to a volume to reduce noise and detail.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 GaussianBlur3DFilter()

```
GaussianBlur3DFilter::GaussianBlur3DFilter (  
    int size,  
    double stdev) [inline]
```

Constructs a [GaussianBlur3DFilter](#) object.

Parameters

<i>size</i>	The size of the Gaussian kernel.
<i>stdev</i>	The standard deviation controlling the blur strength.

4.7.3 Member Function Documentation

4.7.3.1 apply()

```
void GaussianBlur3DFilter::apply (  
    Volume & volume) [override], [virtual]
```

Applies the 3D Gaussian blur operation to the volume.

Parameters

<i>volume</i>	Reference to the 3D volume object to be processed.
---------------	--

Implements [Filter3D](#).

4.7.3.2 setIndex()

```
void GaussianBlur3DFilter::setIndex (  
    int axIndex) [inline]
```

Sets the axis index for the convolution pass.

Parameters

<i>axIndex</i>	The integer index representing the specific axis or slice.
----------------	--

4.7.3.3 setPivot()

```
void GaussianBlur3DFilter::setPivot (
    char pivot) [inline]
```

Sets the pivot axis for the separable convolution.

Parameters

<i>pivot</i>	The character representing the axis (e.g., 'x', 'y', 'z').
--------------	--

The documentation for this class was generated from the following files:

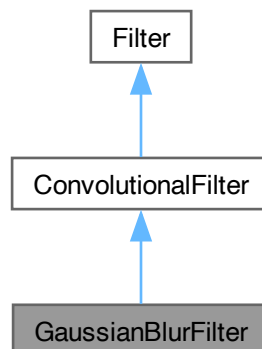
- [Filters3D.h](#)
- [Filters3D.cpp](#)

4.8 GaussianBlurFilter Class Reference

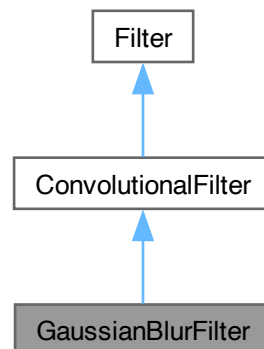
Gaussian blur filter. Uses a Gaussian function to calculate weights for smoothing, which preserves edge details better than a simple box blur.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for GaussianBlurFilter:



Collaboration diagram for GaussianBlurFilter:



Public Member Functions

- [GaussianBlurFilter](#) (int size, double stdev)
Constructs a [GaussianBlurFilter](#) object.
- void [apply](#) ([Image](#) &image) override
Applies the Gaussian blur operation to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.8.1 Detailed Description

Gaussian blur filter. Uses a Gaussian function to calculate weights for smoothing, which preserves edge details better than a simple box blur.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 GaussianBlurFilter()

```
GaussianBlurFilter::GaussianBlurFilter (  
    int size,  
    double stdev)
```

Constructs a [GaussianBlurFilter](#) object.

Parameters

<i>size</i>	The size of the convolution kernel.
<i>stdev</i>	The standard deviation, which controls the extent of the blur.

4.8.3 Member Function Documentation

4.8.3.1 `apply()`

```
void GaussianBlurFilter::apply (  
    Image & image) [override], [virtual]
```

Applies the Gaussian blur operation to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

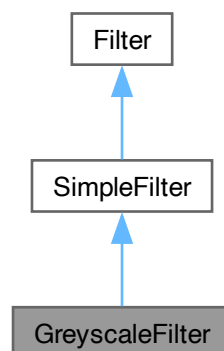
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.9 GreyscaleFilter Class Reference

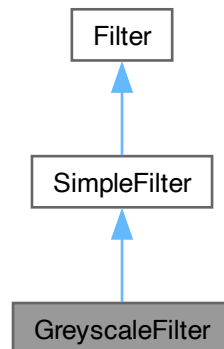
Converts a color image to grayscale.

```
#include <SimpleFilter.h>
```

Inheritance diagram for GreyscaleFilter:



Collaboration diagram for GreyscaleFilter:



Public Member Functions

- void [apply](#) ([Image](#) &image) override
Applies the grayscale conversion to the image.

Public Member Functions inherited from [SimpleFilter](#)

- virtual [~SimpleFilter](#) ()
Default virtual destructor.

4.9.1 Detailed Description

Converts a color image to grayscale.

4.9.2 Member Function Documentation

4.9.2.1 [apply\(\)](#)

```
void GreyscaleFilter::apply (
    Image & image) [override], [virtual]
```

Applies the grayscale conversion to the image.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

Implements [SimpleFilter](#).

The documentation for this class was generated from the following files:

- [SimpleFilter.h](#)
- [SimpleFilter.cpp](#)

4.10 HSLPixel Struct Reference

Represents a single pixel in the HSL (Hue, Saturation, Lightness) colour space.

```
#include <Image.h>
```

Public Attributes

- float [h](#)
Hue angle in degrees [0, 360).
- float [s](#)
Saturation [0.0, 1.0].
- float [l](#)
Lightness [0.0, 1.0].

4.10.1 Detailed Description

Represents a single pixel in the HSL (Hue, Saturation, Lightness) colour space.

4.10.2 Member Data Documentation

4.10.2.1 [h](#)

```
float HSLPixel::h
```

Hue angle in degrees [0, 360).

4.10.2.2 [l](#)

```
float HSLPixel::l
```

Lightness [0.0, 1.0].

4.10.2.3 [s](#)

```
float HSLPixel::s
```

Saturation [0.0, 1.0].

The documentation for this struct was generated from the following file:

- [Image.h](#)

4.11 HSVPixel Struct Reference

Represents a single pixel in the HSV (Hue, Saturation, Value) colour space.

```
#include <Image.h>
```

Public Attributes

- float [h](#)
Hue angle in degrees [0, 360).
- float [s](#)
Saturation [0.0, 1.0].
- float [v](#)
Value/Brightness [0.0, 1.0].

4.11.1 Detailed Description

Represents a single pixel in the HSV (Hue, Saturation, Value) colour space.

4.11.2 Member Data Documentation

4.11.2.1 [h](#)

```
float HSVPixel::h
```

Hue angle in degrees [0, 360).

4.11.2.2 [s](#)

```
float HSVPixel::s
```

Saturation [0.0, 1.0].

4.11.2.3 [v](#)

```
float HSVPixel::v
```

Value/Brightness [0.0, 1.0].

The documentation for this struct was generated from the following file:

- [Image.h](#)

4.12 Image Class Reference

A concrete class for storing and manipulating image data.

```
#include <Image.h>
```

Public Member Functions

- [Image](#) ()
Default constructor. Initializes an empty image.
- [Image](#) (int w, int h, int c)
Constructs a blank (zeroed) image with specified dimensions and channels.
- [Image](#) (const unsigned char *rawData, int w, int h, int srcChannels)
Constructs an image from a raw pixel buffer.
- bool [save](#) (const std::string &filename) const
Saves the current image to disk.
- void [removeAlpha](#) ()
Removes the alpha channel from the image, converting it to 3 channels (RGB).
- int [getWidth](#) () const
Gets the width of the image.
- int [getHeight](#) () const
Gets the height of the image.
- int [getChannels](#) () const
Gets the number of channels in the image.
- void [setChannels](#) (int c)
Sets the number of channels for the image.
- unsigned char [getPixel](#) (int x, int y, int c=0) const
Gets the pixel value at specific coordinates, with edge-clamping.
- void [setPixel](#) (int x, int y, int c, unsigned char value)
Sets the pixel value at specific coordinates.
- float [getPixelAsFloat](#) (int x, int y, int c=0) const
Gets the pixel value as a float, with edge-clamping.
- void [setPixelClamped](#) (int x, int y, int c, int value)
Sets a pixel using an integer value, clamping it to [0, 255] before storing.
- std::vector< unsigned char > & [getData](#) ()
Gets a mutable reference to the internal raw pixel data.
- const std::vector< unsigned char > & [getData](#) () const
Gets a constant reference to the internal raw pixel data.
- void [loadFromData](#) (const unsigned char *rawData, int w, int h, int srcChannels)
Replaces all internal data from a raw buffer.
- std::unique_ptr< [Image](#) > [clone](#) () const
Creates a deep copy of the image.
- void [stripAlphaChannel](#) ()
In-place version of alpha channel stripping for existing 4-channel images.
- std::vector< [HSLPixel](#) > [toHSL](#) () const
Converts the entire image to an array of HSL pixels.
- std::vector< [HSVPixel](#) > [toHSV](#) () const
Converts the entire image to an array of HSV pixels.
- void [fromHSL](#) (const std::vector< [HSLPixel](#) > &hslData)
Replaces the image data by converting an array of HSL pixels back to RGB.
- void [fromHSV](#) (const std::vector< [HSVPixel](#) > &hsvData)
Replaces the image data by converting an array of HSV pixels back to RGB.

Static Public Member Functions

- static `std::unique_ptr< Image > load` (const `std::string` &filename)
Static factory: loads a file and returns a new Image.
- static `std::vector< unsigned char > stripAlphaChannel` (const unsigned char *rgbaData, int pixelCount)
Utility to strip the alpha channel from raw RGBA data.
- static `HSLPixel RGBtoHSL` (unsigned char r, unsigned char g, unsigned char b)
Converts an RGB pixel to HSL colour space.
- static `RGBPixel HSLtoRGB` (const `HSLPixel` &hsl)
Converts an HSL pixel back to RGB colour space.
- static `HSVPixel RGBtoHSV` (unsigned char r, unsigned char g, unsigned char b)
Converts an RGB pixel to HSV colour space.
- static `RGBPixel HSVtoRGB` (const `HSVPixel` &hsv)
Converts an HSV pixel back to RGB colour space.

4.12.1 Detailed Description

A concrete class for storing and manipulating image data.

Manages image dimensions, channel counts (1 for grayscale, 3 for RGB), and raw pixel data. It includes utilities for file I/O, alpha channel stripping, and bulk colour-space conversions.

4.12.2 Constructor & Destructor Documentation

4.12.2.1 Image() [1/3]

```
Image::Image ()
```

Default constructor. Initializes an empty image.

4.12.2.2 Image() [2/3]

```
Image::Image (
    int w,
    int h,
    int c)
```

Constructs a blank (zeroed) image with specified dimensions and channels.

Parameters

<i>w</i>	The width of the image.
<i>h</i>	The height of the image.
<i>c</i>	The number of channels (1 or 3).

4.12.2.3 Image() [3/3]

```
Image::Image (
    const unsigned char * rawData,
    int w,
    int h,
    int srcChannels)
```

Constructs an image from a raw pixel buffer.

If `srcChannels == 4`, the alpha channel is silently stripped to 3-ch RGB.

Parameters

<i>rawData</i>	Pointer to the raw image data.
<i>w</i>	The width of the image.
<i>h</i>	The height of the image.
<i>srcChannels</i>	The number of channels in the source data.

4.12.3 Member Function Documentation

4.12.3.1 clone()

```
std::unique_ptr< Image > Image::clone () const
```

Creates a deep copy of the image.

Returns

A unique pointer to the newly cloned [Image](#).

4.12.3.2 fromHSL()

```
void Image::fromHSL (
    const std::vector< HSLPixel > & hslData)
```

Replaces the image data by converting an array of HSL pixels back to RGB.

Parameters

<i>hslData</i>	The vector of HSLPixels to convert and load.
----------------	--

4.12.3.3 fromHSV()

```
void Image::fromHSV (
    const std::vector< HSVPixel > & hsvData)
```

Replaces the image data by converting an array of HSV pixels back to RGB.

Parameters

<i>hsvData</i>	The vector of HSVPixels to convert and load.
----------------	--

4.12.3.4 getChannels()

```
int Image::getChannels () const
```

Gets the number of channels in the image.

Returns

The channel count (e.g., 1 or 3).

4.12.3.5 getData() [1/2]

```
std::vector< unsigned char > & Image::getData ()
```

Gets a mutable reference to the internal raw pixel data.

Returns

A reference to the underlying std::vector.

4.12.3.6 getData() [2/2]

```
const std::vector< unsigned char > & Image::getData () const
```

Gets a constant reference to the internal raw pixel data.

Returns

A const reference to the underlying std::vector.

4.12.3.7 getHeight()

```
int Image::getHeight () const
```

Gets the height of the image.

Returns

The height in pixels.

4.12.3.8 `getPixel()`

```
unsigned char Image::getPixel (
    int x,
    int y,
    int c = 0) const
```

Gets the pixel value at specific coordinates, with edge-clamping.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>c</i>	The channel index (defaults to 0).

Returns

The unsigned char value of the pixel channel.

4.12.3.9 `getPixelAsFloat()`

```
float Image::getPixelAsFloat (
    int x,
    int y,
    int c = 0) const
```

Gets the pixel value as a float, with edge-clamping.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>c</i>	The channel index (defaults to 0).

Returns

The float value in the range [0.0f, 255.0f].

4.12.3.10 `getWidth()`

```
int Image::getWidth () const
```

Gets the width of the image.

Returns

The width in pixels.

4.12.3.11 HSLtoRGB()

```
RGBPixel Image::HSLtoRGB (  
    const HSLPixel & hsl) [static]
```

Converts an HSL pixel back to RGB colour space.

Parameters

<i>hsl</i>	The HSLPixel to convert.
------------	--

Returns

The converted [RGBPixel](#).

4.12.3.12 HSVtoRGB()

```
RGBPixel Image::HSVtoRGB (  
    const HSVPixel & hsv) [static]
```

Converts an HSV pixel back to RGB colour space.

Parameters

<i>hsv</i>	The HSVPixel to convert.
------------	--

Returns

The converted [RGBPixel](#).

4.12.3.13 load()

```
std::unique_ptr< Image > Image::load (  
    const std::string & filename) [static]
```

Static factory: loads a file and returns a new [Image](#).

Parameters

<i>filename</i>	The path to the image file to load.
-----------------	-------------------------------------

Returns

A unique pointer to the loaded [Image](#), or nullptr on failure.

4.12.3.14 loadFromData()

```
void Image::loadFromData (
    const unsigned char * rawData,
    int w,
    int h,
    int srcChannels)
```

Replaces all internal data from a raw buffer.

Follows the same rules as the 4-argument constructor.

Parameters

<i>rawData</i>	Pointer to the new raw image data.
<i>w</i>	The new width.
<i>h</i>	The new height.
<i>srcChannels</i>	The number of channels in the source data.

4.12.3.15 removeAlpha()

```
void Image::removeAlpha ()
```

Removes the alpha channel from the image, converting it to 3 channels (RGB).

4.12.3.16 RGBtoHSL()

```
HSLPixel Image::RGBtoHSL (
    unsigned char r,
    unsigned char g,
    unsigned char b) [static]
```

Converts an RGB pixel to HSL colour space.

Parameters

<i>r</i>	Red value (0-255).
<i>g</i>	Green value (0-255).
<i>b</i>	Blue value (0-255).

Returns

The converted [HSLPixel](#).

4.12.3.17 RGBtoHSV()

```
HSVPixel Image::RGBtoHSV (
    unsigned char r,
    unsigned char g,
    unsigned char b) [static]
```

Converts an RGB pixel to HSV colour space.

Parameters

<i>r</i>	Red value (0-255).
<i>g</i>	Green value (0-255).
<i>b</i>	Blue value (0-255).

Returns

The converted [HSVPixel](#).

4.12.3.18 save()

```
bool Image::save (
    const std::string & filename) const
```

Saves the current image to disk.

The output format (PNG / JPG / BMP) is chosen by the file extension.

Parameters

<i>filename</i>	The path where the image should be saved.
-----------------	---

Returns

True if the save was successful, false otherwise.

4.12.3.19 setChannels()

```
void Image::setChannels (
    int c)
```

Sets the number of channels for the image.

Parameters

<i>c</i>	The new channel count.
----------	------------------------

4.12.3.20 setPixel()

```
void Image::setPixel (
    int x,
    int y,
    int c,
    unsigned char value)
```

Sets the pixel value at specific coordinates.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>c</i>	The channel index.
<i>value</i>	The value to set (0-255).

4.12.3.21 setPixelClamped()

```
void Image::setPixelClamped (
    int x,
    int y,
    int c,
    int value)
```

Sets a pixel using an integer value, clamping it to [0, 255] before storing.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>c</i>	The channel index.
<i>value</i>	The integer value to store (will be clamped).

4.12.3.22 stripAlphaChannel() [1/2]

```
void Image::stripAlphaChannel ()
```

In-place version of alpha channel stripping for existing 4-channel images.

4.12.3.23 stripAlphaChannel() [2/2]

```
std::vector< unsigned char > Image::stripAlphaChannel (
    const unsigned char * rgbaData,
    int pixelCount) [static]
```

Utility to strip the alpha channel from raw RGBA data.

Parameters

<i>rgbaData</i>	Pointer to the raw 4-channel data.
<i>pixelCount</i>	The total number of pixels.

Returns

A vector containing the 3-channel RGB data.

4.12.3.24 toHSL()

```
std::vector< HSLPixel > Image::toHSL () const
```

Converts the entire image to an array of HSL pixels.

Only valid when channels == 3; throws `std::runtime_error` otherwise.

Returns

A vector of [HSLPixel](#) data representing the image.

4.12.3.25 toHSV()

```
std::vector< HSVPixel > Image::toHSV () const
```

Converts the entire image to an array of HSV pixels.

Only valid when channels == 3; throws `std::runtime_error` otherwise.

Returns

A vector of [HSVPixel](#) data representing the image.

The documentation for this class was generated from the following files:

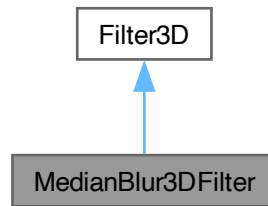
- [Image.h](#)
- [Image.cpp](#)

4.13 MedianBlur3DFilter Class Reference

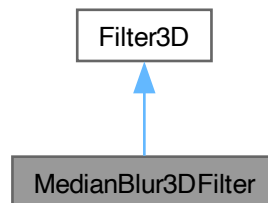
3D Median blur filter.

```
#include <Filters3D.h>
```

Inheritance diagram for MedianBlur3DFilter:



Collaboration diagram for MedianBlur3DFilter:



Public Member Functions

- [MedianBlur3DFilter](#) (int size)
Constructs a [MedianBlur3DFilter](#) object.
- void [apply](#) ([Volume](#) &volume) override
Applies the 3D median blur operation to the volume.

Public Member Functions inherited from [Filter3D](#)

- virtual [~Filter3D](#) ()
Default virtual destructor.

4.13.1 Detailed Description

3D Median blur filter.

Applies a 3D median filter to remove noise (like salt-and-pepper) while preserving edges in volumetric data.

4.13.2 Constructor & Destructor Documentation

4.13.2.1 MedianBlur3DFilter()

```
MedianBlur3DFilter::MedianBlur3DFilter (  
    int size) [explicit]
```

Constructs a [MedianBlur3DFilter](#) object.

Parameters

<i>size</i>	The size of the median filtering window (typically an odd number).
-------------	--

4.13.3 Member Function Documentation

4.13.3.1 apply()

```
void MedianBlur3DFilter::apply (  
    Volume & volume) [override], [virtual]
```

Applies the 3D median blur operation to the volume.

Parameters

<i>volume</i>	Reference to the 3D volume object to be processed.
---------------	--

Implements [Filter3D](#).

The documentation for this class was generated from the following files:

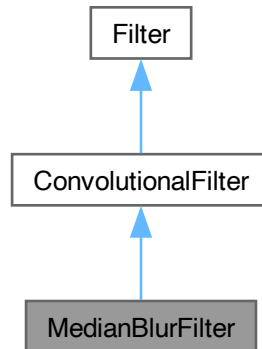
- [Filters3D.h](#)
- [Filters3D.cpp](#)

4.14 MedianBlurFilter Class Reference

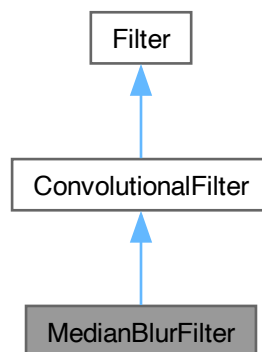
Median blur filter. Replaces each pixel with the median value of its neighboring pixels. Highly effective for removing salt-and-pepper noise.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for MedianBlurFilter:



Collaboration diagram for MedianBlurFilter:



Public Member Functions

- [MedianBlurFilter](#) (int size)
Constructs a [MedianBlurFilter](#) object.
- void [apply](#) ([Image](#) &image) override
Applies the median blur operation to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()

Default virtual destructor.

4.14.1 Detailed Description

Median blur filter. Replaces each pixel with the median value of its neighboring pixels. Highly effective for removing salt-and-pepper noise.

4.14.2 Constructor & Destructor Documentation

4.14.2.1 MedianBlurFilter()

```
MedianBlurFilter::MedianBlurFilter (  
    int size)
```

Constructs a [MedianBlurFilter](#) object.

Parameters

<i>size</i>	The size of the window (usually an odd number).
-------------	---

4.14.3 Member Function Documentation

4.14.3.1 apply()

```
void MedianBlurFilter::apply (  
    Image & image) [override], [virtual]
```

Applies the median blur operation to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

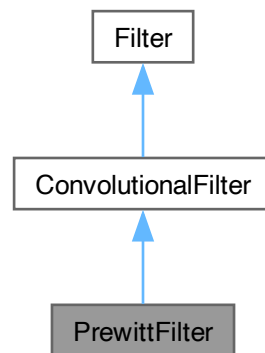
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.15 PrewittFilter Class Reference

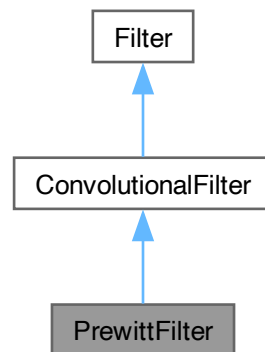
Prewitt edge detection filter. Uses the Prewitt operator for edge detection. Suitable for images with noticeable edges and some noise.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for PrewittFilter:



Collaboration diagram for PrewittFilter:



Public Member Functions

- void `apply` (`Image` &image) override
Applies Prewitt edge detection to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()

Default virtual destructor.

4.15.1 Detailed Description

Prewitt edge detection filter. Uses the Prewitt operator for edge detection. Suitable for images with noticeable edges and some noise.

4.15.2 Member Function Documentation

4.15.2.1 `apply()`

```
void PrewittFilter::apply (
    Image & image) [override], [virtual]
```

Applies Prewitt edge detection to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.16 Projection Class Reference

A utility class for generating 2D image projections from a 3D volume.

```
#include <Projection.h>
```

Static Public Member Functions

- static [Image](#) [maxIntensity](#) (const [Volume](#) &volume, int zStart=-1, int zEnd=-1)
Performs a Maximum Intensity [Projection](#) (MIP) on the volume.
- static [Image](#) [minIntensity](#) (const [Volume](#) &volume, int zStart=-1, int zEnd=-1)
Performs a Minimum Intensity [Projection](#) (MinIP) on the volume.
- static [Image](#) [averageIntensity](#) (const [Volume](#) &volume, int zStart=-1, int zEnd=-1)
Performs an Average Intensity [Projection](#) (AIP) on the volume.
- static [Image](#) [medianIntensity](#) (const [Volume](#) &volume, int zStart=-1, int zEnd=-1)
Performs a Median Intensity [Projection](#) on the volume.

4.16.1 Detailed Description

A utility class for generating 2D image projections from a 3D volume.

All projection methods are static. They reduce volumetric data along the depth (Z-axis) into a single 2D image based on specific statistical criteria (e.g., maximum, minimum, mean, median).

4.16.2 Member Function Documentation

4.16.2.1 averageIntensity()

```
Image Projection::averageIntensity (
    const Volume & volume,
    int zStart = -1,
    int zEnd = -1) [static]
```

Performs an Average Intensity [Projection](#) (AIP) on the volume.

Calculates the mean intensity value of all voxels along the viewing ray for each pixel in the 2D image.

Parameters

<i>volume</i>	The 3D volume to project.
<i>zStart</i>	The starting Z-index (inclusive). Defaults to -1.
<i>zEnd</i>	The ending Z-index (inclusive). Defaults to -1.

Returns

A 2D [Image](#) representing the average intensity projection.

4.16.2.2 maxIntensity()

```
Image Projection::maxIntensity (
    const Volume & volume,
    int zStart = -1,
    int zEnd = -1) [static]
```

Performs a Maximum Intensity [Projection](#) (MIP) on the volume.

Projects the voxel with the highest intensity value along the viewing ray onto the 2D image. Often used for visualizing bright structures like blood vessels.

Parameters

<i>volume</i>	The 3D volume to project.
<i>zStart</i>	The starting Z-index (inclusive) of the slab. Defaults to -1 (full volume).
<i>zEnd</i>	The ending Z-index (inclusive) of the slab. Defaults to -1 (full volume).

Returns

A 2D [Image](#) representing the maximum intensity projection.

4.16.2.3 medianIntensity()

```
Image Projection::medianIntensity (  
    const Volume & volume,  
    int zStart = -1,  
    int zEnd = -1) [static]
```

Performs a Median Intensity [Projection](#) on the volume.

Finds the median intensity value of all voxels along the viewing ray for each pixel in the 2D image.

Parameters

<i>volume</i>	The 3D volume to project.
<i>zStart</i>	The starting Z-index (inclusive). Defaults to -1.
<i>zEnd</i>	The ending Z-index (inclusive). Defaults to -1.

Returns

A 2D [Image](#) representing the median intensity projection.

4.16.2.4 minIntensity()

```
Image Projection::minIntensity (  
    const Volume & volume,  
    int zStart = -1,  
    int zEnd = -1) [static]
```

Performs a Minimum Intensity [Projection](#) (MinIP) on the volume.

Projects the voxel with the lowest intensity value along the viewing ray onto the 2D image. Often used for visualizing dark structures like airways.

Parameters

<i>volume</i>	The 3D volume to project.
<i>zStart</i>	The starting Z-index (inclusive). Defaults to -1.
<i>zEnd</i>	The ending Z-index (inclusive). Defaults to -1.

Returns

A 2D [Image](#) representing the minimum intensity projection.

The documentation for this class was generated from the following files:

- [Projection.h](#)
- [Projection.cpp](#)

4.17 RGBPixel Struct Reference

Represents a single pixel in the RGB colour space.

```
#include <Image.h>
```

Public Attributes

- unsigned char [r](#)
Red channel value (0-255).
- unsigned char [g](#)
Green channel value (0-255).
- unsigned char [b](#)
Blue channel value (0-255).

4.17.1 Detailed Description

Represents a single pixel in the RGB colour space.

4.17.2 Member Data Documentation

4.17.2.1 [b](#)

```
unsigned char RGBPixel::b
```

Blue channel value (0-255).

4.17.2.2 [g](#)

```
unsigned char RGBPixel::g
```

Green channel value (0-255).

4.17.2.3 [r](#)

```
unsigned char RGBPixel::r
```

Red channel value (0-255).

The documentation for this struct was generated from the following file:

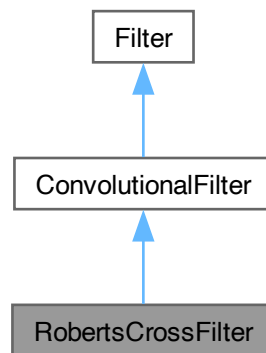
- [Image.h](#)

4.18 RobertsCrossFilter Class Reference

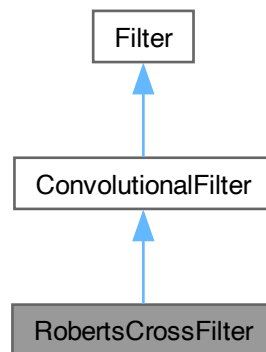
Roberts Cross edge detection filter. Approximates gradient magnitude using diagonal directional differences. Computationally fast.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for RobertsCrossFilter:



Collaboration diagram for RobertsCrossFilter:



Public Member Functions

- void [apply](#) ([Image](#) &image) override
Applies Roberts Cross edge detection to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.18.1 Detailed Description

Roberts Cross edge detection filter. Approximates gradient magnitude using diagonal directional differences. Computationally fast.

4.18.2 Member Function Documentation

4.18.2.1 `apply()`

```
void RobertsCrossFilter::apply (
    Image & image) [override], [virtual]
```

Applies Roberts Cross edge detection to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

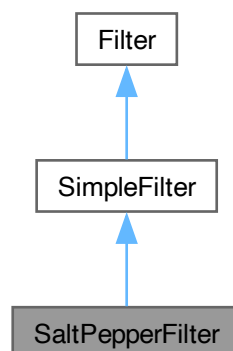
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.19 SaltPepperFilter Class Reference

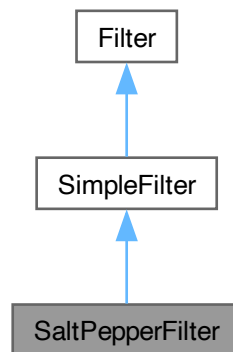
Injects random salt-and-pepper noise into an image.

```
#include <SimpleFilter.h>
```

Inheritance diagram for SaltPepperFilter:



Collaboration diagram for SaltPepperFilter:



Public Member Functions

- [SaltPepperFilter](#) (int amount)
Constructs a [SaltPepperFilter](#).
- void [apply](#) ([Image](#) &image) override
Applies the salt-and-pepper noise to the image.

Public Member Functions inherited from [SimpleFilter](#)

- virtual [~SimpleFilter](#) ()
Default virtual destructor.

4.19.1 Detailed Description

Injects random salt-and-pepper noise into an image.

4.19.2 Constructor & Destructor Documentation

4.19.2.1 SaltPepperFilter()

```
SaltPepperFilter::SaltPepperFilter (  
    int amount)
```

Constructs a [SaltPepperFilter](#).

Parameters

<i>amount</i>	Determines how many pixels are affected by the noise.
---------------	---

4.19.3 Member Function Documentation

4.19.3.1 `apply()`

```
void SaltPepperFilter::apply (  
    Image & image) [override], [virtual]
```

Applies the salt-and-pepper noise to the image.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

Implements [SimpleFilter](#).

The documentation for this class was generated from the following files:

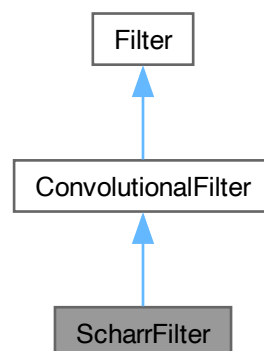
- [SimpleFilter.h](#)
- [SimpleFilter.cpp](#)

4.20 ScharrFilter Class Reference

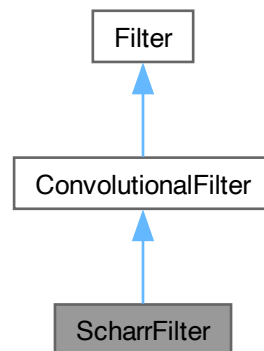
Scharr edge detection filter. Provides a more accurate, rotation-invariant gradient calculation compared to the Sobel operator.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for ScharrFilter:



Collaboration diagram for ScharrFilter:



Public Member Functions

- void [apply](#) ([Image](#) &image) override
Applies Scharr edge detection to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.20.1 Detailed Description

Scharr edge detection filter. Provides a more accurate, rotation-invariant gradient calculation compared to the Sobel operator.

4.20.2 Member Function Documentation

4.20.2.1 [apply\(\)](#)

```
void ScharrFilter::apply (
    Image & image) [override], [virtual]
```

Applies Scharr edge detection to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

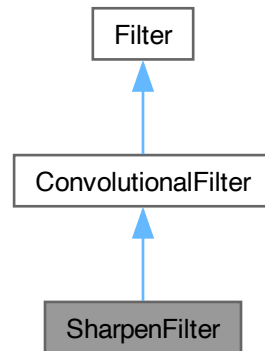
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.21 SharpenFilter Class Reference

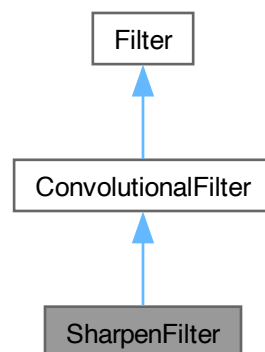
Sharpen filter. Enhances the edges and detail contrast of the image.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for SharpenFilter:



Collaboration diagram for SharpenFilter:



Public Member Functions

- void [apply](#) ([Image](#) &image) override
Applies the sharpen operation to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.21.1 Detailed Description

Sharpen filter. Enhances the edges and detail contrast of the image.

4.21.2 Member Function Documentation

4.21.2.1 `apply()`

```
void SharpenFilter::apply (
    Image & image) [override], [virtual]
```

Applies the sharpen operation to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

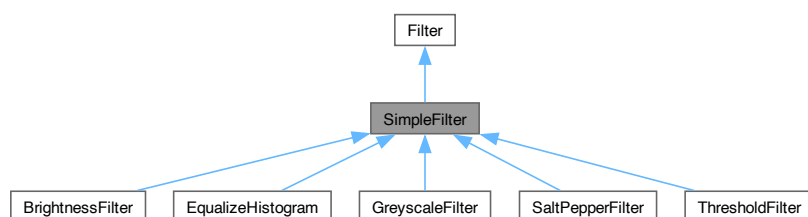
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.22 SimpleFilter Class Reference

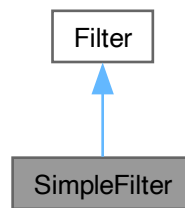
Abstract base class for simple, point-process image filters.

```
#include <SimpleFilter.h>
```

Inheritance diagram for SimpleFilter:



Collaboration diagram for SimpleFilter:



Public Member Functions

- virtual [~SimpleFilter](#) ()
Default virtual destructor.
- virtual void [apply](#) ([Image](#) &image) override=0
Applies the simple filter to the given image.

4.22.1 Detailed Description

Abstract base class for simple, point-process image filters.

Inherits from the base Filter class. Subclasses must implement the [apply\(\)](#) method for pixel-by-pixel operations.

4.22.2 Constructor & Destructor Documentation

4.22.2.1 ~SimpleFilter()

```
virtual SimpleFilter::~~SimpleFilter () [inline], [virtual]
```

Default virtual destructor.

4.22.3 Member Function Documentation

4.22.3.1 apply()

```
virtual void SimpleFilter::apply (  
    Image & image) [override], [pure virtual]
```

Applies the simple filter to the given image.

Parameters

<i>image</i>	Reference to the Image object to be processed in-place.
--------------	---

Implemented in [BrightnessFilter](#), [EqualizeHistogram](#), [GreyscaleFilter](#), [SaltPepperFilter](#), and [ThresholdFilter](#).

The documentation for this class was generated from the following file:

- [SimpleFilter.h](#)

4.23 Slice Class Reference

A utility class for extracting 2D images from a 3D volume.

```
#include <Slice.h>
```

Static Public Member Functions

- static [Image slice](#) (const [Volume](#) &vol, int coord, char pivot)
Extracts a 2D slice from the volume based on a user-provided coordinate.

4.23.1 Detailed Description

A utility class for extracting 2D images from a 3D volume.

Contains static methods to slice through a 3D volumetric dataset along a specified axis, returning the result as a standard 2D [Image](#).

4.23.2 Member Function Documentation

4.23.2.1 slice()

```
Image Slice::slice (
    const Volume & vol,
    int coord,
    char pivot) [static]
```

Extracts a 2D slice from the volume based on a user-provided coordinate.

Parameters

<i>vol</i>	The 3D volume to slice.
<i>coord</i>	The 1-based coordinate index for the slice.
<i>pivot</i>	The axis to fix ('X' for YZ plane, 'Y' for XZ plane).

Returns

A 2D [Image](#) representing the extracted slice.

The documentation for this class was generated from the following files:

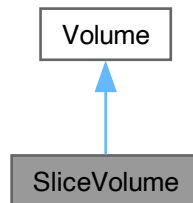
- [Slice.h](#)
- [Slice.cpp](#)

4.24 SliceVolume Class Reference

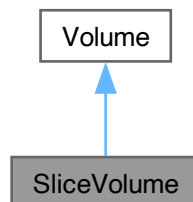
Represents a 3D volume that supports plane rotation and slice extraction.

```
#include <SliceVolume.h>
```

Inheritance diagram for SliceVolume:



Collaboration diagram for SliceVolume:



Public Member Functions

- void [rotate](#) (int coord, int kernSize, char plane)
Rotates the active plane of the volume for subsequent slicing.
- [Image slice](#) (int coord)
Extracts a 2D slice from the volume along the currently rotated plane.

Public Member Functions inherited from [Volume](#)

- [Volume](#) ()
Default constructor. Initializes an empty volume.
- [Volume](#) (int w, int h, int d, int c=1)
Constructs a volume with specified dimensions, initialized with zeroes.

- virtual `~Volume ()`
Virtual destructor for the `Volume` class.
- bool `load` (const std::string &prefix, int first=-1, int last=-1, std::string extension=".png")
Loads a volume from a sequence of image files.
- int `getWidth ()` const
Gets the width (x dimension) of the volume.
- int `getHeight ()` const
Gets the height (y dimension) of the volume.
- int `getDepth ()` const
Gets the depth (z dimension) of the volume.
- int `getChannels ()` const
Gets the number of channels per voxel.
- unsigned char `getVoxel` (int x, int y, int z, int c=0) const
Retrieves the voxel value at the specified coordinates and channel.
- void `setVoxel` (int x, int y, int z, int c, unsigned char value)
Sets the voxel value at the specified coordinates and channel.
- void `setVoxel` (int x, int y, int z, unsigned char value)
Convenience method to set the voxel value on channel 0.
- float `getVoxelAsFloat` (int x, int y, int z, int c=0) const
Retrieves the voxel value as a float.
- void `setVoxelClamped` (int x, int y, int z, int c, int value)
Sets the voxel value using an integer, clamping it to valid bounds.
- void `setVoxelClamped` (int x, int y, int z, int value)
Convenience method to set a clamped integer voxel value on channel 0.
- std::vector< unsigned char > & `getData ()`
Gets a mutable reference to the internal raw voxel data.
- const std::vector< unsigned char > & `getData ()` const
Gets a constant reference to the internal raw voxel data.
- std::unique_ptr< `Volume` > `clone ()` const
Creates a deep copy of the volume.

Additional Inherited Members

Protected Attributes inherited from `Volume`

- int `width`
The x dimension (width) of the volume.
- int `height`
The y dimension (height) of the volume.
- int `depth`
The z dimension (depth or number of slices).
- int `channels`
The number of channels per voxel (1 = greyscale, 3 = RGB).
- std::vector< unsigned char > `data`
The flat 1D vector storing the actual volumetric data.

4.24.1 Detailed Description

Represents a 3D volume that supports plane rotation and slice extraction.

Extends the basic `Volume` data structure with methods specifically designed to change the orientation (plane) and extract corresponding 2D Images.

4.24.2 Member Function Documentation

4.24.2.1 rotate()

```
void SliceVolume::rotate (
    int coord,
    int kernSize,
    char plane)
```

Rotates the active plane of the volume for subsequent slicing.

Parameters

<i>coord</i>	The coordinate index around which the rotation is applied.
<i>kernSize</i>	The size of the kernel or neighborhood involved in the rotation.
<i>plane</i>	The character representing the new active plane (e.g., 'X', 'Y', 'Z').

4.24.2.2 slice()

```
Image SliceVolume::slice (
    int coord)
```

Extracts a 2D slice from the volume along the currently rotated plane.

Parameters

<i>coord</i>	The coordinate index at which to extract the slice.
--------------	---

Returns

A 2D [Image](#) representing the extracted slice.

The documentation for this class was generated from the following files:

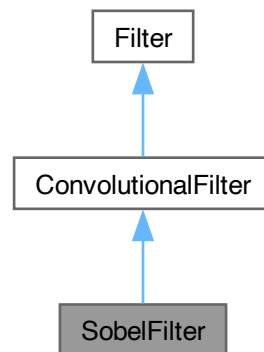
- [SliceVolume.h](#)
- [SliceVolume.cpp](#)

4.25 SobelFilter Class Reference

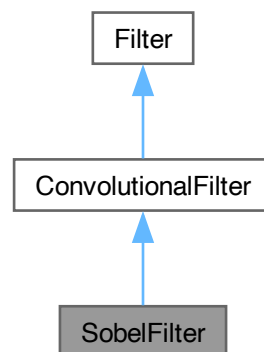
Sobel edge detection filter. Highlights edges by calculating the spatial gradient of the image intensity using the Sobel operator.

```
#include <ConvolutionalFilter.h>
```

Inheritance diagram for SobelFilter:



Collaboration diagram for SobelFilter:



Public Member Functions

- void [apply](#) ([Image](#) &image) override
Applies Sobel edge detection to the image.

Public Member Functions inherited from [ConvolutionalFilter](#)

- virtual [~ConvolutionalFilter](#) ()
Default virtual destructor.

4.25.1 Detailed Description

Sobel edge detection filter. Highlights edges by calculating the spatial gradient of the image intensity using the Sobel operator.

4.25.2 Member Function Documentation

4.25.2.1 `apply()`

```
void SobelFilter::apply (  
    Image & image) [override], [virtual]
```

Applies Sobel edge detection to the image.

Parameters

<i>image</i>	Reference to the image object to be processed.
--------------	--

Implements [ConvolutionalFilter](#).

The documentation for this class was generated from the following files:

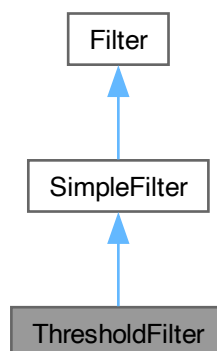
- [ConvolutionalFilter.h](#)
- [ConvolutionalFilter.cpp](#)

4.26 ThresholdFilter Class Reference

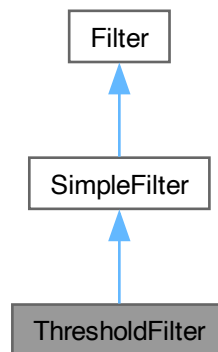
Applies a threshold to an image, converting it to binary (black and white).

```
#include <SimpleFilter.h>
```

Inheritance diagram for ThresholdFilter:



Collaboration diagram for ThresholdFilter:



Public Member Functions

- [ThresholdFilter](#) (int threshold, const std::string &type="HSV")
Constructs a [ThresholdFilter](#).
- void [apply](#) ([Image](#) &image) override
Applies the binary thresholding to the image.

Public Member Functions inherited from [SimpleFilter](#)

- virtual [~SimpleFilter](#) ()
Default virtual destructor.

4.26.1 Detailed Description

Applies a threshold to an image, converting it to binary (black and white).

4.26.2 Constructor & Destructor Documentation

4.26.2.1 ThresholdFilter()

```
ThresholdFilter::ThresholdFilter (  
    int threshold,  
    const std::string & type = "HSV")
```

Constructs a [ThresholdFilter](#).

Parameters

<i>threshold</i>	The cutoff value. Pixels below this become black, others white.
<i>type</i>	The color model to use for brightness evaluation (defaults to "HSV").

4.26.3 Member Function Documentation**4.26.3.1 apply()**

```
void ThresholdFilter::apply (
    Image & image) [override], [virtual]
```

Applies the binary thresholding to the image.

Parameters

<i>image</i>	Reference to the Image object to be processed.
--------------	--

Implements [SimpleFilter](#).

The documentation for this class was generated from the following files:

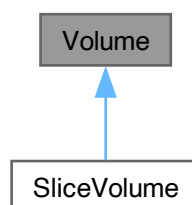
- [SimpleFilter.h](#)
- [SimpleFilter.cpp](#)

4.27 Volume Class Reference

A class representing a 3D volume of pixel/voxel data.

```
#include <Volume.h>
```

Inheritance diagram for Volume:



Public Member Functions

- [Volume](#) ()
Default constructor. Initializes an empty volume.
- [Volume](#) (int w, int h, int d, int c=1)
Constructs a volume with specified dimensions, initialized with zeroes.
- virtual [~Volume](#) ()
Virtual destructor for the [Volume](#) class.
- bool [load](#) (const std::string &prefix, int first=-1, int last=-1, std::string extension=".png")
Loads a volume from a sequence of image files.
- int [getWidth](#) () const
Gets the width (x dimension) of the volume.
- int [getHeight](#) () const
Gets the height (y dimension) of the volume.
- int [getDepth](#) () const
Gets the depth (z dimension) of the volume.
- int [getChannels](#) () const
Gets the number of channels per voxel.
- unsigned char [getVoxel](#) (int x, int y, int z, int c=0) const
Retrieves the voxel value at the specified coordinates and channel.
- void [setVoxel](#) (int x, int y, int z, int c, unsigned char value)
Sets the voxel value at the specified coordinates and channel.
- void [setVoxel](#) (int x, int y, int z, unsigned char value)
Convenience method to set the voxel value on channel 0.
- float [getVoxelAsFloat](#) (int x, int y, int z, int c=0) const
Retrieves the voxel value as a float.
- void [setVoxelClamped](#) (int x, int y, int z, int c, int value)
Sets the voxel value using an integer, clamping it to valid bounds.
- void [setVoxelClamped](#) (int x, int y, int z, int value)
Convenience method to set a clamped integer voxel value on channel 0.
- std::vector< unsigned char > & [getData](#) ()
Gets a mutable reference to the internal raw voxel data.
- const std::vector< unsigned char > & [getData](#) () const
Gets a constant reference to the internal raw voxel data.
- std::unique_ptr< [Volume](#) > [clone](#) () const
Creates a deep copy of the volume.

Protected Attributes

- int [width](#)
The x dimension (width) of the volume.
- int [height](#)
The y dimension (height) of the volume.
- int [depth](#)
The z dimension (depth or number of slices).
- int [channels](#)
The number of channels per voxel (1 = greyscale, 3 = RGB).
- std::vector< unsigned char > [data](#)
The flat 1D vector storing the actual volumetric data.

4.27.1 Detailed Description

A class representing a 3D volume of pixel/voxel data.

Manages a flat 1D buffer that conceptually represents a 3D grid of voxels, supporting both single-channel (grayscale) and multi-channel (RGB) data.

4.27.2 Constructor & Destructor Documentation

4.27.2.1 Volume() [1/2]

```
Volume::Volume ()
```

Default constructor. Initializes an empty volume.

4.27.2.2 Volume() [2/2]

```
Volume::Volume (
    int w,
    int h,
    int d,
    int c = 1)
```

Constructs a volume with specified dimensions, initialized with zeroes.

Parameters

<i>w</i>	The width (x dimension).
<i>h</i>	The height (y dimension).
<i>d</i>	The depth (z dimension).
<i>c</i>	The number of channels (defaults to 1).

4.27.2.3 ~Volume()

```
Volume::~Volume () [virtual]
```

Virtual destructor for the [Volume](#) class.

4.27.3 Member Function Documentation

4.27.3.1 clone()

```
std::unique_ptr< Volume > Volume::clone () const
```

Creates a deep copy of the volume.

Returns

A unique pointer to the newly cloned [Volume](#).

4.27.3.2 getChannels()

```
int Volume::getChannels () const
```

Gets the number of channels per voxel.

Returns

The channel count (e.g., 1 or 3).

4.27.3.3 getData() [1/2]

```
std::vector< unsigned char > & Volume::getData ()
```

Gets a mutable reference to the internal raw voxel data.

Returns

A reference to the underlying std::vector.

4.27.3.4 getData() [2/2]

```
const std::vector< unsigned char > & Volume::getData () const
```

Gets a constant reference to the internal raw voxel data.

Returns

A const reference to the underlying std::vector.

4.27.3.5 getDepth()

```
int Volume::getDepth () const
```

Gets the depth (z dimension) of the volume.

Returns

The depth in voxels (number of slices).

4.27.3.6 getHeight()

```
int Volume::getHeight () const
```

Gets the height (y dimension) of the volume.

Returns

The height in voxels.

4.27.3.7 getVoxel()

```
unsigned char Volume::getVoxel (
    int x,
    int y,
    int z,
    int c = 0) const
```

Retrieves the voxel value at the specified coordinates and channel.

Out of bounds x/y/z coordinates are clamped to the nearest valid index.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>z</i>	The z-coordinate.
<i>c</i>	The channel index (defaults to 0).

Returns

The unsigned char value of the voxel channel.

4.27.3.8 getVoxelAsFloat()

```
float Volume::getVoxelAsFloat (
    int x,
    int y,
    int z,
    int c = 0) const
```

Retrieves the voxel value as a float.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>z</i>	The z-coordinate.
<i>c</i>	The channel index (defaults to 0).

Returns

The float value in the range [0.0, 255.0].

4.27.3.9 getWidth()

```
int Volume::getWidth () const
```

Gets the width (x dimension) of the volume.

Returns

The width in voxels.

4.27.3.10 load()

```
bool Volume::load (
    const std::string & prefix,
    int first = -1,
    int last = -1,
    std::string extension = ".png")
```

Loads a volume from a sequence of image files.

Parameters

<i>prefix</i>	The file path prefix (e.g., "Scans/TestVolume/vol").
<i>first</i>	The starting slice index. Defaults to -1 (auto-detect).
<i>last</i>	The ending slice index. Defaults to -1 (auto-detect).
<i>extension</i>	The image file extension (e.g., ".png").

Returns

True if the volume was successfully loaded, false otherwise.

4.27.3.11 setVoxel() [1/2]

```
void Volume::setVoxel (
    int x,
    int y,
    int z,
    int c,
    unsigned char value)
```

Sets the voxel value at the specified coordinates and channel.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>z</i>	The z-coordinate.
<i>c</i>	The channel index.
<i>value</i>	The unsigned char value to set.

4.27.3.12 setVoxel() [2/2]

```
void Volume::setVoxel (
    int x,
    int y,
    int z,
    unsigned char value)
```

Convenience method to set the voxel value on channel 0.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>z</i>	The z-coordinate.
<i>value</i>	The unsigned char value to set.

4.27.3.13 setVoxelClamped() [1/2]

```
void Volume::setVoxelClamped (
    int x,
    int y,
    int z,
    int c,
    int value)
```

Sets the voxel value using an integer, clamping it to valid bounds.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>z</i>	The z-coordinate.
<i>c</i>	The channel index.
<i>value</i>	The integer value (will be clamped to [0, 255] before storing).

4.27.3.14 setVoxelClamped() [2/2]

```
void Volume::setVoxelClamped (
    int x,
    int y,
    int z,
    int value)
```

Convenience method to set a clamped integer voxel value on channel 0.

Parameters

<i>x</i>	The x-coordinate.
<i>y</i>	The y-coordinate.
<i>z</i>	The z-coordinate.
<i>value</i>	The integer value to clamp and set.

4.27.4 Member Data Documentation

4.27.4.1 channels

```
int Volume::channels [protected]
```

The number of channels per voxel (1 = greyscale, 3 = RGB).

4.27.4.2 data

```
std::vector<unsigned char> Volume::data [protected]
```

The flat 1D vector storing the actual volumetric data.

4.27.4.3 depth

```
int Volume::depth [protected]
```

The z dimension (depth or number of slices).

4.27.4.4 height

```
int Volume::height [protected]
```

The y dimension (height) of the volume.

4.27.4.5 width

```
int Volume::width [protected]
```

The x dimension (width) of the volume.

The documentation for this class was generated from the following files:

- [Volume.h](#)
- [Volume.cpp](#)

Chapter 5

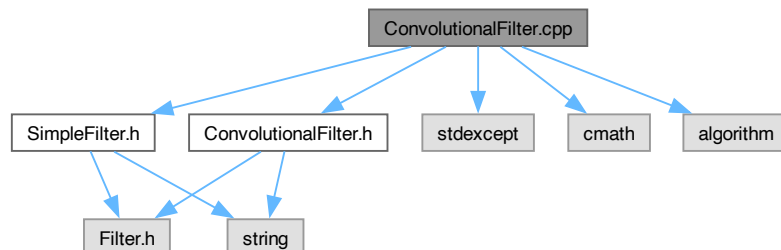
File Documentation

5.1 ConvolutionalFilter.cpp File Reference

Implementation of various convolutional image filters.

```
#include "ConvolutionalFilter.h"  
#include "SimpleFilter.h"  
#include <stdexcept>  
#include <cmath>  
#include <algorithm>
```

Include dependency graph for ConvolutionalFilter.cpp:



Functions

- `std::vector< float > calculateGaussianKernel (int size, float sigma)`
- `void EdgeDetection (Image &image, const std::vector< std::vector< int > > &Gx, const std::vector< std::vector< int > > &Gy, double max_magnitude)`

5.1.1 Detailed Description

Implementation of various convolutional image filters.

This file implements the methods for filters such as BoxBlur, GaussianBlur, MedianBlur, Sharpen, edge detection (Sobel, Prewitt, Scharr, RobertsCross), and Emboss.

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: @ada-sy1124)
Akira C T Eisenbeiss (GitHub: @ada-ace25)
Ju Lin (GitHub: @ada-jl4025)
Yichen Liu (GitHub: @ada-yl2425)
Charli Maguire (GitHub: @ada-cm1625)
Jiacheng Zhao (GitHub: @ada-jz1225)
Siqi Yao (GitHub: @ada-sy325)

5.1.2 Function Documentation

5.1.2.1 calculateGaussianKernel()

```
std::vector< float > calculateGaussianKernel (  
    int size,  
    float sigma)
```

5.1.2.2 EdgeDetection()

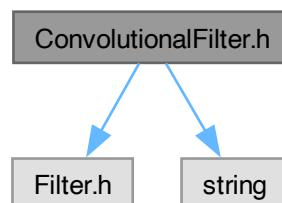
```
void EdgeDetection (  
    Image & image,  
    const std::vector< std::vector< int > > & Gx,  
    const std::vector< std::vector< int > > & Gy,  
    double max_magnitude)
```

5.2 ConvolutionalFilter.h File Reference

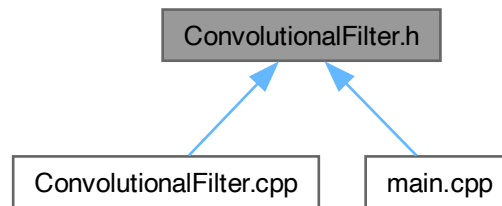
Defines a collection of convolutional image filters.

```
#include "Filter.h"  
#include <string>
```

Include dependency graph for ConvolutionalFilter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ConvolutionalFilter](#)
Base class for convolutional filters. Inherits from `Filter` and provides an abstract interface for all image processing operations based on convolution matrices (kernels).
- class [BoxBlurFilter](#)
Box blur (mean blur) filter. Smooths the image by replacing each pixel with the average value of its local neighborhood.
- class [GaussianBlurFilter](#)
Gaussian blur filter. Uses a Gaussian function to calculate weights for smoothing, which preserves edge details better than a simple box blur.
- class [MedianBlurFilter](#)
Median blur filter. Replaces each pixel with the median value of its neighboring pixels. Highly effective for removing salt-and-pepper noise.
- class [SharpenFilter](#)
Sharpen filter. Enhances the edges and detail contrast of the image.
- class [SobelFilter](#)
Sobel edge detection filter. Highlights edges by calculating the spatial gradient of the image intensity using the Sobel operator.
- class [PrewittFilter](#)
Prewitt edge detection filter. Uses the Prewitt operator for edge detection. Suitable for images with noticeable edges and some noise.
- class [ScharrFilter](#)
Scharr edge detection filter. Provides a more accurate, rotation-invariant gradient calculation compared to the Sobel operator.
- class [RobertsCrossFilter](#)
Roberts Cross edge detection filter. Approximates gradient magnitude using diagonal directional differences. Computationally fast.
- class [EmbossFilter](#)
Emboss effect filter. Creates a 3D embossed visual effect by replacing pixels with shadows cast from a specific direction.

5.2.1 Detailed Description

Defines a collection of convolutional image filters.

This file contains the abstract [ConvolutionalFilter](#) base class and several concrete implementations for spatial domain image filtering operations, such as blurring, sharpening, edge detection, and embossing using convolution kernels.

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: @ada-sy1124)
Akira C T Eisenbeiss (GitHub: @ada-ace25)
Ju Lin (GitHub: @ada-jl4025)
Yichen Liu (GitHub: @ada-yl2425)
Charli Maguire (GitHub: @ada-cm1625)
Jiacheng Zhao (GitHub: @ada-jz1225)
Siqi Yao (GitHub: @ada-sy325)

5.3 ConvolutionalFilter.h

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef CONVOLUTIONALFILTER_H
00018 #define CONVOLUTIONALFILTER_H
00019
00020 #include "Filter.h"
00021 #include <string>
00022
00028 class ConvolutionalFilter : public Filter {
00029 public:
00033     virtual ~ConvolutionalFilter() {}
00034
00039     virtual void apply(Image& image) override = 0;
00040 };
00041
00046 class BoxBlurFilter : public ConvolutionalFilter {
00047 private:
00048     int size;
00049
00050 public:
00055     BoxBlurFilter(int size);
00056
00061     void apply(Image& image) override;
00062 };
00063
00069 class GaussianBlurFilter : public ConvolutionalFilter {
00070 private:
00071     int size;
00072     double stdev;
00073
00074 public:
00080     GaussianBlurFilter(int size, double stdev);
00081
00086     void apply(Image& image) override;
00087 };
00088
00094 class MedianBlurFilter : public ConvolutionalFilter {
00095 private:
00096     int size;
00097
00098 public:
00103     MedianBlurFilter(int size);
00104
00109     void apply(Image& image) override;
00110 };
00111
00116 class SharpenFilter : public ConvolutionalFilter {
00117 public:
00122     void apply(Image& image) override;
00123 };
00124
00130 class SobelFilter : public ConvolutionalFilter {
00131 public:
00136     void apply(Image& image) override;
00137 };
00138
00144 class PrewittFilter : public ConvolutionalFilter {
00145 public:
00150     void apply(Image& image) override;
00151 };
00152
00158 class ScharrFilter : public ConvolutionalFilter {

```

```

00159 public:
00164     void apply(Image& image) override;
00165 };
00166
00172 class RobertsCrossFilter : public ConvolutionalFilter {
00173 public:
00178     void apply(Image& image) override;
00179 };
00180
00186 class EmbossFilter : public ConvolutionalFilter {
00187 private:
00188     float strength;
00189     std::string direction;
00190     std::string type;
00191
00192 public:
00199     EmbossFilter(float strength, const std::string& direction,
00200                  const std::string& type = "HSV");
00201
00206     void apply(Image& image) override;
00207 };
00208
00209 #endif

```

5.4 Filters3D.cpp File Reference

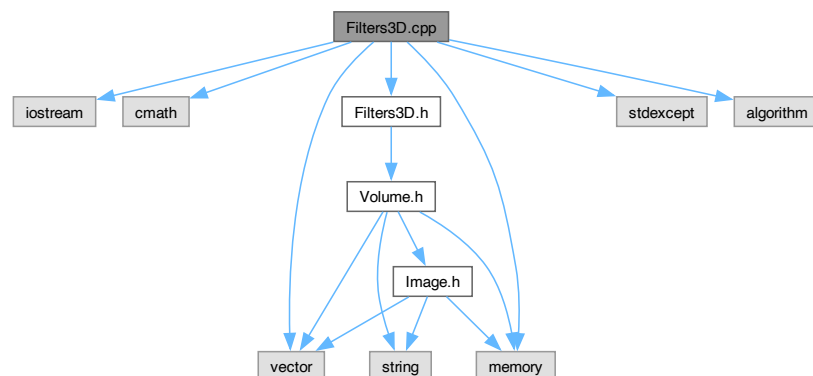
Implementation of 3D volumetric image filters.

```

#include <iostream>
#include <cmath>
#include <vector>
#include <memory>
#include <stdexcept>
#include <algorithm>
#include "Filters3D.h"

```

Include dependency graph for Filters3D.cpp:



5.4.1 Detailed Description

Implementation of 3D volumetric image filters.

This file implements the methods for 3D separable Gaussian blur and 3D Median blur filters applied to volumetric datasets.

Author

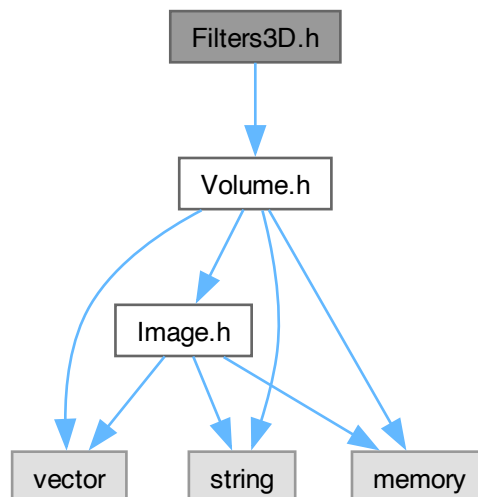
- Group Name: BarnesHut
Siyuan Yuan (GitHub: @ada-sy1124)
Akira C T Eisenbeiss (GitHub: @ada-ace25)
Ju Lin (GitHub: @ada-jl4025)
Yichen Liu (GitHub: @ada-yl2425)
Charli Maguire (GitHub: @ada-cm1625)
Jiacheng Zhao (GitHub: @ada-jz1225)
Siqi Yao (GitHub: @ada-sy325)

5.5 Filters3D.h File Reference

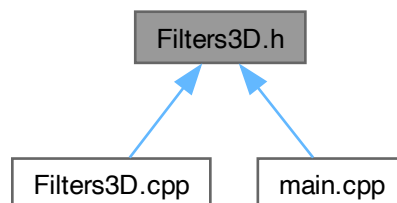
Defines 3D volumetric image filters.

```
#include "Volume.h"
```

Include dependency graph for Filters3D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Filter3D](#)
Base class for separable 3D convolution filters.
- class [GaussianBlur3DFilter](#)
3D Gaussian blur filter.
- class [MedianBlur3DFilter](#)
3D Median blur filter.

5.5.1 Detailed Description

Defines 3D volumetric image filters.

This file contains the abstract [Filter3D](#) base class and concrete implementations for separable 3D Gaussian blurring and 3D Median blurring applied to volumetric data.

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: [@ada-sy1124](#))
Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
Ju Lin (GitHub: [@ada-jl4025](#))
Yichen Liu (GitHub: [@ada-yl2425](#))
Charli Maguire (GitHub: [@ada-cm1625](#))
Jiacheng Zhao (GitHub: [@ada-jz1225](#))
Siqi Yao (GitHub: [@ada-sy325](#))

5.6 Filters3D.h

[Go to the documentation of this file.](#)

```
00001
00016
00017 #ifndef FILTERS3D_H
00018 #define FILTERS3D_H
00019
00020 #include "Volume.h"
00021
00027 class Filter3D {
00028 public:
00032     virtual ~Filter3D() {}
00033
00038     virtual void apply(Volume& volume) = 0;
00039 };
00040
00046 class GaussianBlur3DFilter : public Filter3D {
00047 private:
00048     int size;
00049     double stdev;
00050     char pivot;
00051     int axIndex;
00052
00053 public:
00059     GaussianBlur3DFilter(int size, double stdev) : size(size), stdev(stdev) {}
00060
00065     void apply(Volume& volume) override;
00066
00071     void setPivot(char pivot) {
00072         this->pivot = pivot;
00073     }
00074
00079     void setIndex(int axIndex) {
00080         this->axIndex = axIndex;
00081     }

```

```

00082 };
00083
00090 class MedianBlur3DFilter : public Filter3D {
00091 private:
00092     int size;
00093
00094 public:
00099     explicit MedianBlur3DFilter(int size);
00100
00105     void apply(Volume& volume) override;
00106 };
00107
00108 #endif

```

5.7 Image.cpp File Reference

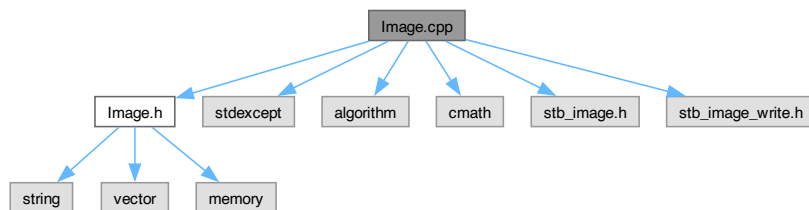
Implementation of the [Image](#) class for image manipulation and I/O.

```

#include "Image.h"
#include <stdexcept>
#include <algorithm>
#include <cmath>
#include "stb_image.h"
#include "stb_image_write.h"

```

Include dependency graph for Image.cpp:



5.7.1 Detailed Description

Implementation of the [Image](#) class for image manipulation and I/O.

This file implements the methods for the [Image](#) class, handling image construction, file loading/saving using `stb_image`, pixel access with edge-clamping, and color space conversions (RGB, HSL, HSV).

Author

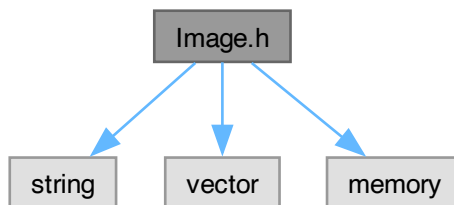
- Group Name: BarnesHut
 Siyuan Yuan (GitHub: [@ada-sy1124](#))
 Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
 Ju Lin (GitHub: [@ada-jl4025](#))
 Yichen Liu (GitHub: [@ada-yl2425](#))
 Charli Maguire (GitHub: [@ada-cm1625](#))
 Jiacheng Zhao (GitHub: [@ada-jz1225](#))
 Siqi Yao (GitHub: [@ada-sy325](#))

5.8 Image.h File Reference

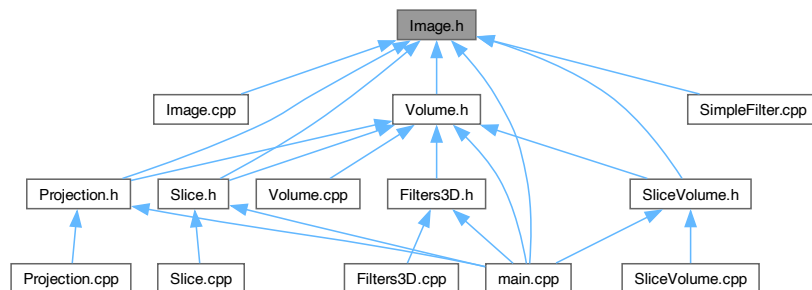
Defines the [Image](#) class and colour-space pixel structures.

```
#include <string>
#include <vector>
#include <memory>
```

Include dependency graph for Image.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [RGBPixel](#)
Represents a single pixel in the RGB colour space.
- struct [HSLPixel](#)
Represents a single pixel in the HSL (Hue, Saturation, Lightness) colour space.
- struct [HSVPixel](#)
Represents a single pixel in the HSV (Hue, Saturation, Value) colour space.
- class [Image](#)
A concrete class for storing and manipulating image data.

5.8.1 Detailed Description

Defines the [Image](#) class and colour-space pixel structures.

This file provides a single concrete [Image](#) class that handles runtime channels (grayscale or RGB), file I/O, pixel access, and colour-space conversions (RGB, HSL, HSV).

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: [@ada-sy1124](#))
Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
Ju Lin (GitHub: [@ada-jl4025](#))
Yichen Liu (GitHub: [@ada-yl2425](#))
Charli Maguire (GitHub: [@ada-cm1625](#))
Jiacheng Zhao (GitHub: [@ada-jz1225](#))
Siqi Yao (GitHub: [@ada-sy325](#))

5.9 Image.h

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef IMAGE_H
00018 #define IMAGE_H
00019
00020 #include <string>
00021 #include <vector>
00022 #include <memory>
00023
00024
00025 // Colour-space pixel structs
00026
00031 struct RGBPixel {
00032     unsigned char r;
00033     unsigned char g;
00034     unsigned char b;
00035 };
00036
00041 struct HSLPixel {
00042     float h;
00043     float s;
00044     float l;
00045 };
00046
00051 struct HSVPixel {
00052     float h;
00053     float s;
00054     float v;
00055 };
00056
00057
00058 // Single concrete Image class.
00059 //
00060 // `channels` is a runtime attribute (1 = grayscale, 3 = RGB).
00061 // No more abstract base / subclass hierarchy -- this avoids the
00062 // factory-loading problem and the GrayscaleFilter mutation bug.
00063
00071 class Image {
00072 private:
00073     int width;
00074     int height;
00075     int channels;
00076     std::vector<unsigned char> data;
00077
00078 public:
00079     // ---- Constructors ----
00080
00084     Image();
00085

```

```

00092     Image(int w, int h, int c);    // blank (zeroed) image with c channels
00093
00102     Image(const unsigned char* rawData, int w, int h, int srcChannels);
00103
00104     // ---- File I/O (uses stb_image / stb_image_write) ----
00105
00111     static std::unique_ptr<Image> load(const std::string& filename);
00112
00119     bool save(const std::string& filename) const;
00120
00124     void removeAlpha();
00125
00126     // Properties
00127
00132     int getWidth() const;
00133
00138     int getHeight() const;
00139
00144     int getChannels() const;
00145
00150     void setChannels(int c);
00151
00152     // Pixel access (unsigned char, with edge-clamp)
00153
00161     unsigned char getPixel(int x, int y, int c = 0) const;
00162
00170     void setPixel(int x, int y, int c, unsigned char value);
00171
00172     // Pixel access -- wider types for filter math
00173
00181     float getPixelAsFloat(int x, int y, int c = 0) const;
00182
00190     void setPixelClamped(int x, int y, int c, int value);
00191
00192
00193
00194     // Raw data access
00195
00200     std::vector<unsigned char>& getData();
00201
00206     const std::vector<unsigned char>& getData() const;
00207
00216     void loadFromData(const unsigned char* rawData, int w, int h, int srcChannels);
00217
00222     std::unique_ptr<Image> clone() const;
00223
00224     // RGBA to RGB utility
00225
00232     static std::vector<unsigned char> stripAlphaChannel(
00233         const unsigned char* rgbaData, int pixelCount);
00234
00238     void stripAlphaChannel(); // in-place version (for existing 4-channel images)
00239
00240
00241     // Single-pixel colour-space conversions (static helpers)
00242
00250     static HSLPixel RGBtoHSL(unsigned char r, unsigned char g, unsigned char b);
00251
00257     static RGBPixel HSLtoRGB(const HSLPixel& hsl);
00258
00266     static HSVPixel RGBtoHSV(unsigned char r, unsigned char g, unsigned char b);
00267
00273     static RGBPixel HSVtoRGB(const HSVPixel& hsv);
00274
00275
00276     // Batch colour-space conversions
00277
00283     std::vector<HSLPixel> toHSL() const;
00284
00290     std::vector<HSVPixel> toHSV() const;
00291
00296     void fromHSL(const std::vector<HSLPixel>& hslData);
00297
00302     void fromHSV(const std::vector<HSVPixel>& hsvData);
00303 };
00304
00305 #endif

```

5.10 main.cpp File Reference

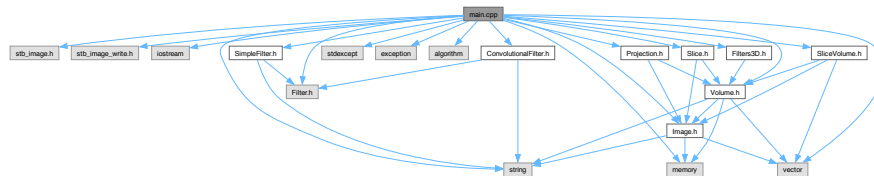
Main entry point for the APLImageFilters application.

```

#include "stb_image.h"
#include "stb_image_write.h"
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <stdexcept>
#include <exception>
#include <algorithm>
#include "Image.h"
#include "Filter.h"
#include "SimpleFilter.h"
#include "ConvolutionalFilter.h"
#include "Filters3D.h"
#include "Volume.h"
#include "Projection.h"
#include "Slice.h"
#include "SliceVolume.h"

```

Include dependency graph for main.cpp:



Macros

- `#define STB_IMAGE_IMPLEMENTATION`
- `#define STB_IMAGE_WRITE_IMPLEMENTATION`

Functions

- `int main (int argc, char *argv[])`

5.10.1 Detailed Description

Main entry point for the APIImageFilters application.

Parses command-line arguments and dispatches execution to either the 2D image processing pipeline or the 3D volume processing pipeline. Handles filter instantiation, error catching, and file I/O operations.

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: @ada-sy1124)
Akira C T Eisenbeiss (GitHub: @ada-ace25)
Ju Lin (GitHub: @ada-jl4025)
Yichen Liu (GitHub: @ada-yl2425)
Charli Maguire (GitHub: @ada-cm1625)
Jiacheng Zhao (GitHub: @ada-jz1225)
Siqi Yao (GitHub: @ada-sy325)

5.10.2 Macro Definition Documentation

5.10.2.1 STB_IMAGE_IMPLEMENTATION

```
#define STB_IMAGE_IMPLEMENTATION
```

APIImageFilters - Advanced Programming Group Project

Main entry point: parses command-line arguments and dispatches to the appropriate image/volume processing pipeline.

5.10.2.2 STB_IMAGE_WRITE_IMPLEMENTATION

```
#define STB_IMAGE_WRITE_IMPLEMENTATION
```

5.10.3 Function Documentation

5.10.3.1 main()

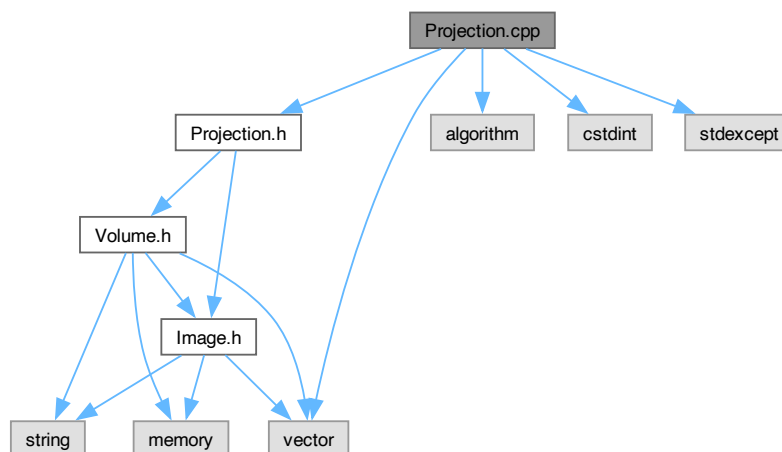
```
int main (
    int argc,
    char * argv[])
```

5.11 Projection.cpp File Reference

Implementation of 2D image projections from 3D volumetric data.

```
#include "Projection.h"
#include <algorithm>
#include <cstdlib>
#include <stdexcept>
#include <vector>
```

Include dependency graph for Projection.cpp:



5.11.1 Detailed Description

Implementation of 2D image projections from 3D volumetric data.

This file implements the methods for projecting a 3D volume (or a specific slab) into a 2D image. It includes Maximum Intensity [Projection](#) (MIP), Minimum Intensity [Projection](#) (MinIP), Average Intensity [Projection](#) (AIP), and a histogram-based Median Intensity [Projection](#).

Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: @ada-sy1124)
 Akira C T Eisenbeiss (GitHub: @ada-ace25)
 Ju Lin (GitHub: @ada-jl4025)
 Yichen Liu (GitHub: @ada-yl2425)
 Charli Maguire (GitHub: @ada-cm1625)
 Jiacheng Zhao (GitHub: @ada-jz1225)
 Siqi Yao (GitHub: @ada-sy325)

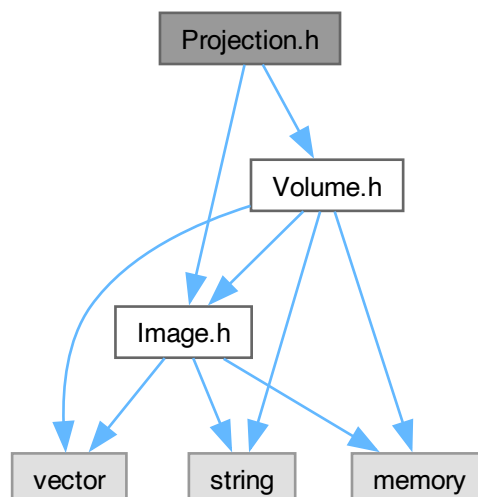
5.12 Projection.h File Reference

Defines operations for creating 2D projections from 3D volumetric data.

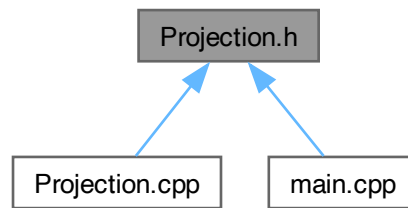
```
#include "Volume.h"
```

```
#include "Image.h"
```

Include dependency graph for Projection.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Projection](#)

A utility class for generating 2D image projections from a 3D volume.

5.12.1 Detailed Description

Defines operations for creating 2D projections from 3D volumetric data.

This file provides a static class, [Projection](#), which contains methods to perform various intensity projections (Maximum, Minimum, Average, and Median) along the Z-axis of a 3D volume, optionally within a specified slice range (slab).

Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: [@ada-sy1124](#))
 Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
 Ju Lin (GitHub: [@ada-jl4025](#))
 Yichen Liu (GitHub: [@ada-yl2425](#))
 Charli Maguire (GitHub: [@ada-cm1625](#))
 Jiacheng Zhao (GitHub: [@ada-jz1225](#))
 Siqi Yao (GitHub: [@ada-sy325](#))

5.13 Projection.h

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef PROJECTION_H
00018 #define PROJECTION_H
00019
00020 #include "Volume.h"
00021 #include "Image.h"
00022
00030 class Projection {
00031 public:
00041     // zStart/zEnd default to -1 meaning "use full volume".
  
```

```

00042 // When specified, they are 0-indexed inclusive bounds for the z slab.
00043 static Image maxIntensity(const Volume& volume, int zStart = -1, int zEnd = -1);
00044
00054 static Image minIntensity(const Volume& volume, int zStart = -1, int zEnd = -1);
00055
00065 static Image averageIntensity(const Volume& volume, int zStart = -1, int zEnd = -1);
00066
00076 static Image medianIntensity(const Volume& volume, int zStart = -1, int zEnd = -1);
00077
00078 private:
00087 static void resolveSlabBounds(int depth, int& zStart, int& zEnd);
00088 };
00089
00090 #endif

```

5.14 SimpleFilter.cpp File Reference

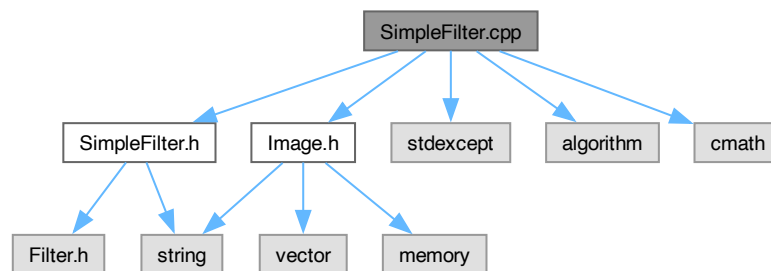
Implementation of simple point-process image filters.

```

#include "SimpleFilter.h"
#include "Image.h"
#include <stdexcept>
#include <algorithm>
#include <cmath>

```

Include dependency graph for SimpleFilter.cpp:



5.14.1 Detailed Description

Implementation of simple point-process image filters.

This file implements various basic image adjustments including grayscale conversion, manual and automatic brightness adjustment, histogram equalization (supporting Grayscale and RGB via HSV/HSL), binary thresholding, and salt-and-pepper noise injection.

Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: @ada-sy1124)
 Akira C T Eisenbeiss (GitHub: @ada-ace25)
 Ju Lin (GitHub: @ada-jl4025)
 Yichen Liu (GitHub: @ada-yl2425)
 Charli Maguire (GitHub: @ada-cm1625)
 Jiacheng Zhao (GitHub: @ada-jz1225)
 Siqi Yao (GitHub: @ada-sy325)

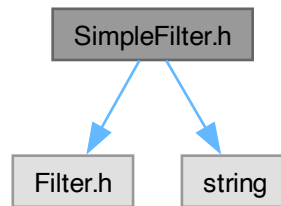
5.15 SimpleFilter.h File Reference

Defines a collection of simple point-process image filters.

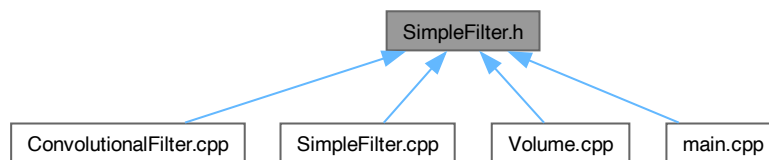
```
#include "Filter.h"
```

```
#include <string>
```

Include dependency graph for SimpleFilter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SimpleFilter](#)
Abstract base class for simple, point-process image filters.
- class [GreyscaleFilter](#)
Converts a color image to grayscale.
- class [BrightnessFilter](#)
Adjusts the brightness of an image.
- class [EqualizeHistogram](#)
Performs histogram equalization to improve image contrast.
- class [ThresholdFilter](#)
Applies a threshold to an image, converting it to binary (black and white).
- class [SaltPepperFilter](#)
Injects random salt-and-pepper noise into an image.

5.15.1 Detailed Description

Defines a collection of simple point-process image filters.

This file contains the abstract [SimpleFilter](#) base class and several concrete implementations for basic image adjustments, such as grayscale conversion, brightness adjustment, histogram equalization, thresholding, and noise injection.

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: [@ada-sy1124](#))
Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
Ju Lin (GitHub: [@ada-jl4025](#))
Yichen Liu (GitHub: [@ada-yl2425](#))
Charli Maguire (GitHub: [@ada-cm1625](#))
Jiacheng Zhao (GitHub: [@ada-jz1225](#))
Siqi Yao (GitHub: [@ada-sy325](#))

5.16 SimpleFilter.h

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef SIMPLEFILTER_H
00018 #define SIMPLEFILTER_H
00019
00020 #include "Filter.h"
00021 #include <string>
00022
00029 class SimpleFilter : public Filter {
00030 public:
00034     virtual ~SimpleFilter() {}
00035
00040     virtual void apply(Image& image) override = 0;
00041 };
00042
00047 class GreyscaleFilter : public SimpleFilter {
00048 public:
00053     void apply(Image& image) override;
00054 };
00055
00062 class BrightnessFilter : public SimpleFilter {
00063 private:
00064     int value;
00065     bool autoMode;
00066
00067 public:
00072     BrightnessFilter(int value);
00073
00077     BrightnessFilter(); // auto mode
00078
00083     void apply(Image& image) override;
00084 };
00085
00090 class EqualizeHistogram : public SimpleFilter {
00091 private:
00092     std::string type;
00093
00098     void equaliseGreyscale(Image& img);
00099
00104     void equaliseRGB(Image& img);
00105
00106 public:
00111     EqualizeHistogram(const std::string& type = "HSV");
00112
00117     void apply(Image& image) override;
00118
00123     void applyRGB(Image& image);
00124

```

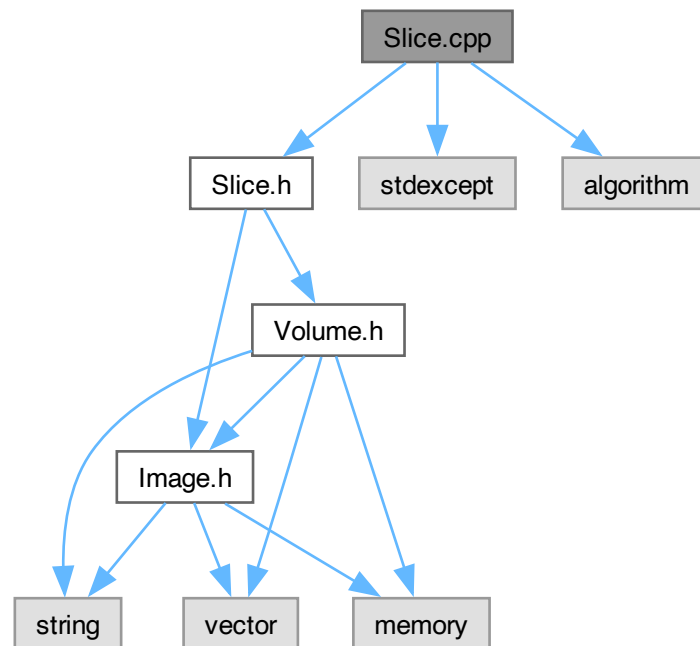
```
00129     void applyGrayscale(Image& image);
00130 };
00131
00136 class ThresholdFilter : public SimpleFilter {
00137 private:
00138     int threshold;
00139     std::string type;    // "HSV" or "HSL" ///< The color space to evaluate brightness in.
00140
00141     // helper functions for different image types
00142
00147     void applyRGB(Image& image);
00148
00153     void applyGrayscale(Image& image);
00154
00155 public:
00161     ThresholdFilter(int threshold, const std::string& type = "HSV");
00162
00167     void apply(Image& image) override;
00168 };
00169
00174 class SaltPepperFilter : public SimpleFilter {
00175 private:
00176     int amount;
00177
00178 public:
00183     SaltPepperFilter(int amount);
00184
00189     void apply(Image& image) override;
00190 };
00191
00192 #endif
```

5.17 Slice.cpp File Reference

Implementation of 2D slice extraction from 3D volumetric data.

```
#include "Slice.h"
#include <stdexcept>
#include <algorithm>
```

Include dependency graph for Slice.cpp:



5.17.1 Detailed Description

Implementation of 2D slice extraction from 3D volumetric data.

This file implements the methods to extract orthogonal 2D slices (e.g., YZ or XZ planes) from a 3D [Volume](#) object, handling coordinate conversion (1-based to 0-based) and orientation mapping.

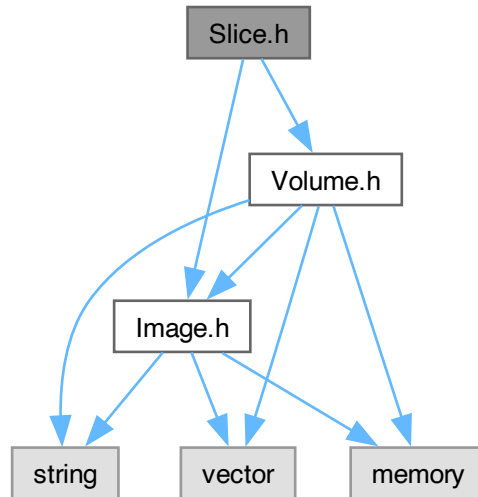
Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: [@ada-sy1124](#))
 Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
 Ju Lin (GitHub: [@ada-jl4025](#))
 Yichen Liu (GitHub: [@ada-yl2425](#))
 Charli Maguire (GitHub: [@ada-cm1625](#))
 Jiacheng Zhao (GitHub: [@ada-jz1225](#))
 Siqi Yao (GitHub: [@ada-sy325](#))

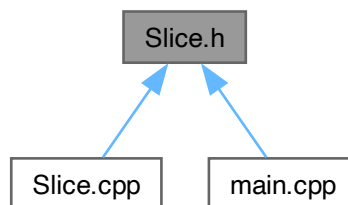
5.18 Slice.h File Reference

Defines operations for extracting 2D slices from 3D volumetric data.

```
#include "Image.h"
#include "Volume.h"
Include dependency graph for Slice.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Slice](#)
A utility class for extracting 2D images from a 3D volume.

5.18.1 Detailed Description

Defines operations for extracting 2D slices from 3D volumetric data.

This file provides a static [Slice](#) class that allows users to extract specific orthogonal 2D planes (e.g., YZ or XZ planes) from a given 3D [Volume](#).

Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: @ada-sy1124)
 Akira C T Eisenbeiss (GitHub: @ada-ace25)
 Ju Lin (GitHub: @ada-jl4025)
 Yichen Liu (GitHub: @ada-yl2425)
 Charli Maguire (GitHub: @ada-cm1625)
 Jiacheng Zhao (GitHub: @ada-jz1225)
 Siqi Yao (GitHub: @ada-sy325)

5.19 Slice.h

[Go to the documentation of this file.](#)

```

00001
00015
00016 #ifndef SLICE_H
00017 #define SLICE_H
00018
00019 #include "Image.h"
00020 #include "Volume.h"
00021
00028 class Slice {
00029 public:
00037     // pivot = 'X' → YZ plane (fix x, user gives x coordinate)
00038     // pivot = 'Y' → XZ plane (fix y, user gives y coordinate)
00039     // coord is 1-based as per spec
00040     static Image slice(const Volume& vol, int coord, char pivot);
00041
00042 private:
00050     static Image extract(const Volume& vol, int fixedIdx, char pivot);
00051 };
00052
00053 #endif

```

5.20 SliceVolume.cpp File Reference

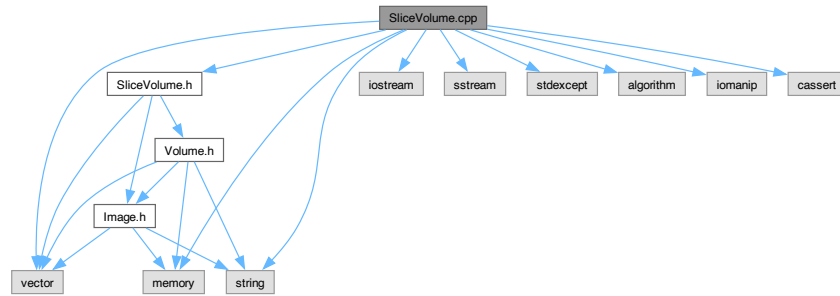
Implementation of the [SliceVolume](#) class for optimized slice extraction.

```

#include "SliceVolume.h"
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
#include <memory>
#include <algorithm>
#include <iomanip>
#include <cassert>

```

Include dependency graph for SliceVolume.cpp:



5.20.1 Detailed Description

Implementation of the [SliceVolume](#) class for optimized slice extraction.

This file implements methods to temporarily rotate and crop a 3D volume to isolate a specific slab (XZ or YZ planes) based on a kernel size. This optimizes memory usage and processing time when applying 3D convolution operations prior to extracting a 2D slice.

Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: [@ada-sy1124](#))
 Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
 Ju Lin (GitHub: [@ada-jl4025](#))
 Yichen Liu (GitHub: [@ada-yl2425](#))
 Charli Maguire (GitHub: [@ada-cm1625](#))
 Jiacheng Zhao (GitHub: [@ada-jz1225](#))
 Siqi Yao (GitHub: [@ada-sy325](#))

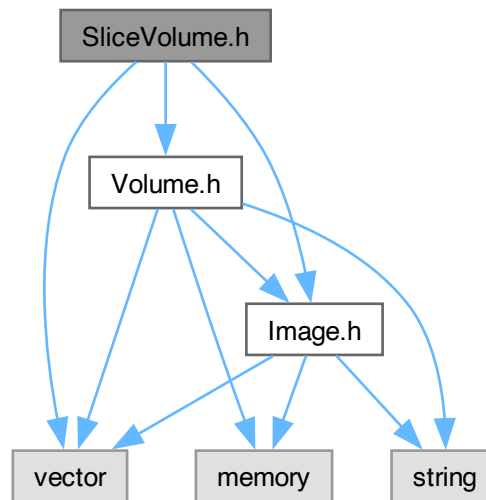
5.21 SliceVolume.h File Reference

Defines a specialized [Volume](#) class for slice manipulation.

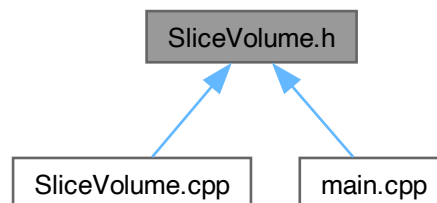
```
#include "Volume.h"
#include "Image.h"
```

```
#include <vector>
```

Include dependency graph for SliceVolume.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SliceVolume](#)

Represents a 3D volume that supports plane rotation and slice extraction.

5.21.1 Detailed Description

Defines a specialized [Volume](#) class for slice manipulation.

Inherits from the base [Volume](#) class to provide functionality for rotating the volumetric dataset and extracting 2D slices along specific planes.

Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: @ada-sy1124)
 Akira C T Eisenbeiss (GitHub: @ada-ace25)
 Ju Lin (GitHub: @ada-jl4025)
 Yichen Liu (GitHub: @ada-yl2425)
 Charli Maguire (GitHub: @ada-cm1625)
 Jiacheng Zhao (GitHub: @ada-jz1225)
 Siqi Yao (GitHub: @ada-sy325)

5.22 SliceVolume.h

[Go to the documentation of this file.](#)

```

00001
00015
00016 #include "Volume.h"
00017 #include "Image.h"
00018 #include <vector>
00019
00026 class SliceVolume : public Volume {
00027 private:
00031     char rotatedPlane = 'X'; // set by rotate()
00032
00033 public:
00040     void rotate(int coord, int kernSize, char plane);
00041
00047     Image slice(int coord);
00048
00049 };

```

5.23 Volume.cpp File Reference

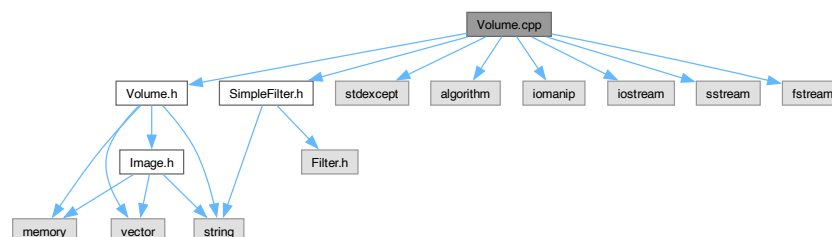
Implementation of the [Volume](#) class for 3D volumetric data management.

```

#include "Volume.h"
#include <stdexcept>
#include <algorithm>
#include <iomanip>
#include "SimpleFilter.h"
#include <iostream>
#include <sstream>
#include <fstream>

```

Include dependency graph for Volume.cpp:



5.23.1 Detailed Description

Implementation of the [Volume](#) class for 3D volumetric data management.

This file implements the methods for the [Volume](#) class, including multi-channel volumetric data allocation, optimized file sequence loading (with zero-padding detection and binary search for bounds), voxel access with bounds clamping, and deep copying.

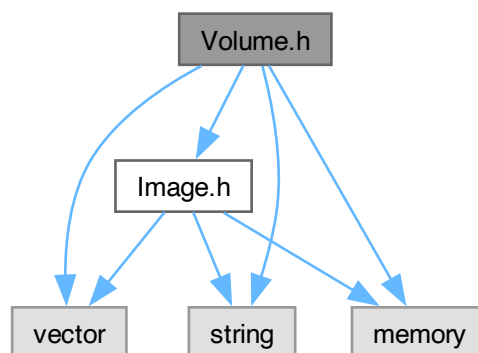
Author

- Group Name: BarnesHut
 Siyuan Yuan (GitHub: [@ada-sy1124](#))
 Akira C T Eisenbeiss (GitHub: [@ada-ace25](#))
 Ju Lin (GitHub: [@ada-jl4025](#))
 Yichen Liu (GitHub: [@ada-yl2425](#))
 Charli Maguire (GitHub: [@ada-cm1625](#))
 Jiacheng Zhao (GitHub: [@ada-jz1225](#))
 Siqi Yao (GitHub: [@ada-sy325](#))

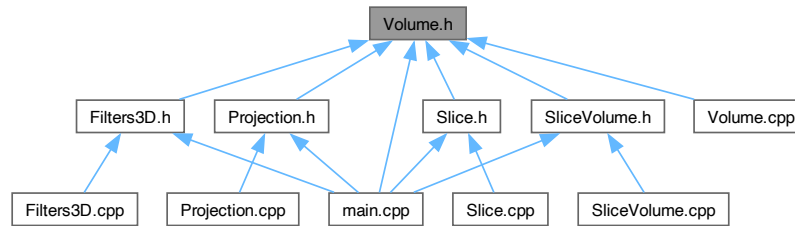
5.24 Volume.h File Reference

Defines the base [Volume](#) class for 3D volumetric data.

```
#include <vector>
#include <string>
#include "Image.h"
#include <memory>
Include dependency graph for Volume.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class `Volume`
A class representing a 3D volume of pixel/voxel data.

5.24.1 Detailed Description

Defines the base `Volume` class for 3D volumetric data.

This file provides the core data structure for representing and manipulating 3D volumes (e.g., loaded from a sequence of 2D image slices). It handles voxel access, clamping, and file I/O operations.

Author

- Group Name: BarnesHut
Siyuan Yuan (GitHub: @ada-sy1124)
Akira C T Eisenbeiss (GitHub: @ada-ace25)
Ju Lin (GitHub: @ada-jl4025)
Yichen Liu (GitHub: @ada-yl2425)
Charli Maguire (GitHub: @ada-cm1625)
Jiacheng Zhao (GitHub: @ada-jz1225)
Siqi Yao (GitHub: @ada-sy325)

5.25 Volume.h

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef VOLUME_H
00018 #define VOLUME_H
00019
00020 #include <vector>
00021 #include <string>
00022 #include "Image.h"
00023 #include <memory> // for std::unique_ptr
00024
00031 class Volume {
00032 protected:
00033     int width;
00034     int height;
00035     int depth;
  
```

```

00036     int channels;
00037
00038     // flat buffer layout: data[((z * height + y) * width + x) * channels + c]
00042     std::vector<unsigned char> data;
00043
00044 public:
00045
00046     // constructors
00050     Volume(); // default = empty volume
00051
00059     Volume(int w, int h, int d, int c = 1); // volume with zeroes
00060
00064     virtual ~Volume(); // destructor
00065
00066     // file input and output
00075     // prefix is e.g. "Scans/TestVolume/vol"
00076     // extension is e.g. ".png"
00077     // first/last are optional slice indices, -1 means auto-detect
00078     bool load(const std::string& prefix,
00079              int first = -1,
00080              int last = -1,
00081              std::string extension = ".png");
00082
00083     // properties
00088     int getWidth() const;
00089
00094     int getHeight() const;
00095
00100     int getDepth() const;
00101
00106     int getChannels() const;
00107
00108     // access voxels -- multi-channel
00118     // out of bounds x/y/z are clamped to nearest valid
00119     unsigned char getVoxel(int x, int y, int z, int c = 0) const;
00120
00129     void setVoxel(int x, int y, int z, int c, unsigned char value);
00130
00138     void setVoxel(int x, int y, int z, unsigned char value); // convenience: channel 0
00139
00148     // return voxel value as float in [0.0, 255.0]
00149     float getVoxelAsFloat(int x, int y, int z, int c = 0) const;
00150
00159     // accepts int, clamps to [0, 255] before storing
00160     void setVoxelClamped(int x, int y, int z, int c, int value);
00161
00169     void setVoxelClamped(int x, int y, int z, int value); // convenience: channel 0
00170
00171     // raw data access
00176     std::vector<unsigned char>& getData();
00177
00182     const std::vector<unsigned char>& getData() const;
00183
00184     // deep copy
00189     std::unique_ptr<Volume> clone() const;
00190
00191
00192 };
00193
00194 #endif

```

Index

- ~ConvolutionalFilter
 - ConvolutionalFilter, [13](#)
- ~Filter3D
 - Filter3D, [19](#)
- ~SimpleFilter
 - SimpleFilter, [54](#)
- ~Volume
 - Volume, [64](#)
- apply
 - BoxBlurFilter, [9](#)
 - BrightnessFilter, [11](#)
 - ConvolutionalFilter, [13](#)
 - EmbossFilter, [15](#)
 - EqualizeHistogram, [17](#)
 - Filter3D, [19](#)
 - GaussianBlur3DFilter, [21](#)
 - GaussianBlurFilter, [24](#)
 - GreyscaleFilter, [25](#)
 - MedianBlur3DFilter, [39](#)
 - MedianBlurFilter, [41](#)
 - PrewittFilter, [43](#)
 - RobertsCrossFilter, [48](#)
 - SaltPepperFilter, [50](#)
 - ScharrFilter, [51](#)
 - SharpenFilter, [53](#)
 - SimpleFilter, [54](#)
 - SobelFilter, [60](#)
 - ThresholdFilter, [62](#)
- applyGrayscale
 - EqualizeHistogram, [17](#)
- applyRGB
 - EqualizeHistogram, [18](#)
- averageIntensity
 - Projection, [44](#)
- b
 - RGBPixel, [46](#)
- BoxBlurFilter, [7](#)
 - apply, [9](#)
 - BoxBlurFilter, [8](#)
- BrightnessFilter, [9](#)
 - apply, [11](#)
 - BrightnessFilter, [10](#), [11](#)
- calculateGaussianKernel
 - ConvolutionalFilter.cpp, [72](#)
- channels
 - Volume, [69](#)
- clone
 - Image, [30](#)
 - Volume, [64](#)
- ConvolutionalFilter, [11](#)
 - ~ConvolutionalFilter, [13](#)
 - apply, [13](#)
- ConvolutionalFilter.cpp, [71](#)
 - calculateGaussianKernel, [72](#)
 - EdgeDetection, [72](#)
- ConvolutionalFilter.h, [72](#)
- data
 - Volume, [69](#)
- depth
 - Volume, [69](#)
- EdgeDetection
 - ConvolutionalFilter.cpp, [72](#)
- EmbossFilter, [13](#)
 - apply, [15](#)
 - EmbossFilter, [15](#)
- EqualizeHistogram, [16](#)
 - apply, [17](#)
 - applyGrayscale, [17](#)
 - applyRGB, [18](#)
 - EqualizeHistogram, [17](#)
- Filter3D, [18](#)
 - ~Filter3D, [19](#)
 - apply, [19](#)
- Filters3D.cpp, [75](#)
- Filters3D.h, [76](#)
- fromHSL
 - Image, [30](#)
- fromHSV
 - Image, [30](#)
- g
 - RGBPixel, [46](#)
- GaussianBlur3DFilter, [20](#)
 - apply, [21](#)
 - GaussianBlur3DFilter, [21](#)
 - setIndex, [21](#)
 - setPivot, [21](#)
- GaussianBlurFilter, [22](#)
 - apply, [24](#)
 - GaussianBlurFilter, [23](#)
- getChannels
 - Image, [31](#)
 - Volume, [64](#)
- getData

- Image, 31
 - Volume, 65
- getDepth
 - Volume, 65
- getHeight
 - Image, 31
 - Volume, 65
- getPixel
 - Image, 31
- getPixelAsFloat
 - Image, 32
- getVoxel
 - Volume, 65
- getVoxelAsFloat
 - Volume, 66
- getWidth
 - Image, 32
 - Volume, 66
- GreyscaleFilter, 24
 - apply, 25
- h
 - HSLPixel, 26
 - HSVPixel, 27
- height
 - Volume, 69
- HSLPixel, 26
 - h, 26
 - l, 26
 - s, 26
- HSLtoRGB
 - Image, 32
- HSVPixel, 27
 - h, 27
 - s, 27
 - v, 27
- HSVtoRGB
 - Image, 33
- Image, 28
 - clone, 30
 - fromHSL, 30
 - fromHSV, 30
 - getChannels, 31
 - getData, 31
 - getHeight, 31
 - getPixel, 31
 - getPixelAsFloat, 32
 - getWidth, 32
 - HSLtoRGB, 32
 - HSVtoRGB, 33
 - Image, 29
 - load, 33
 - loadFromData, 33
 - removeAlpha, 34
 - RGBtoHSL, 34
 - RGBtoHSV, 34
 - save, 35
 - setChannels, 35
 - setPixel, 35
 - setPixelClamped, 36
 - stripAlphaChannel, 36
 - toHSL, 37
 - toHSV, 37
- Image.cpp, 78
- Image.h, 79
- l
 - HSLPixel, 26
- load
 - Image, 33
 - Volume, 66
- loadFromData
 - Image, 33
- main
 - main.cpp, 83
- main.cpp, 81
 - main, 83
 - STB_IMAGE_IMPLEMENTATION, 83
 - STB_IMAGE_WRITE_IMPLEMENTATION, 83
- maxIntensity
 - Projection, 44
- MedianBlur3DFilter, 38
 - apply, 39
- MedianBlur3DFilter, 39
- MedianBlurFilter, 40
 - apply, 41
- MedianBlurFilter, 41
- medianIntensity
 - Projection, 44
- minIntensity
 - Projection, 45
- PrewittFilter, 42
 - apply, 43
- Projection, 43
 - averageIntensity, 44
 - maxIntensity, 44
 - medianIntensity, 44
 - minIntensity, 45
- Projection.cpp, 83
- Projection.h, 84
- r
 - RGBPixel, 46
- removeAlpha
 - Image, 34
- RGBPixel, 46
 - b, 46
 - g, 46
 - r, 46
- RGBtoHSL
 - Image, 34
- RGBtoHSV
 - Image, 34
- RobertsCrossFilter, 47
 - apply, 48

- rotate
 - SliceVolume, 58
- s
 - HSLPixel, 26
 - HSVPixel, 27
- SaltPepperFilter, 48
 - apply, 50
 - SaltPepperFilter, 49
- save
 - Image, 35
- ScharrFilter, 50
 - apply, 51
- setChannels
 - Image, 35
- setIndex
 - GaussianBlur3DFilter, 21
- setPivot
 - GaussianBlur3DFilter, 21
- setPixel
 - Image, 35
- setPixelClamped
 - Image, 36
- setVoxel
 - Volume, 67
- setVoxelClamped
 - Volume, 68
- SharpenFilter, 52
 - apply, 53
- SimpleFilter, 53
 - ~SimpleFilter, 54
 - apply, 54
- SimpleFilter.cpp, 86
- SimpleFilter.h, 87
- Slice, 55
 - slice, 55
- slice
 - Slice, 55
 - SliceVolume, 58
- Slice.cpp, 89
- Slice.h, 90
- SliceVolume, 56
 - rotate, 58
 - slice, 58
- SliceVolume.cpp, 92
- SliceVolume.h, 93
- SobelFilter, 58
 - apply, 60
- STB_IMAGE_IMPLEMENTATION
 - main.cpp, 83
- STB_IMAGE_WRITE_IMPLEMENTATION
 - main.cpp, 83
- stripAlphaChannel
 - Image, 36
- ThresholdFilter, 60
 - apply, 62
 - ThresholdFilter, 61
- toHSL
 - Image, 37
- toHSV
 - Image, 37
- v
 - HSVPixel, 27
- Volume, 62
 - ~Volume, 64
 - channels, 69
 - clone, 64
 - data, 69
 - depth, 69
 - getChannels, 64
 - getData, 65
 - getDepth, 65
 - getHeight, 65
 - getVoxel, 65
 - getVoxelAsFloat, 66
 - getWidth, 66
 - height, 69
 - load, 66
 - setVoxel, 67
 - setVoxelClamped, 68
 - Volume, 64
 - width, 69
- Volume.cpp, 95
- Volume.h, 96
- width
 - Volume, 69